TECHPULSE: A REAL-TIME HYBRID RETRIEVAL-AUGMENTED GENERATION SYSTEM FOR EMERGING TECHNOLOGY INTELLIGENCE

Progress Report

by

Aye Khin Khin Hpone (Yolanda Lim) st125970

Dechathon Niamsa-Ard st126235

AT82.9002 Selected Topic: Data Engineering and MLOps

Instructor: Dr. Chantri Polprasert

Teaching Assistants: Swaraj Bhanja, Suryansh Srivastava

Asian Institute of Technology

School of Engineering and Technology

Thailand

March 2026

ABSTRACT

The rapid expansion of emerging technologies has created a fragmented knowledge ecosystem where scholarly research coexists with fast-moving industry discussions. Large language models (LLMs) offer advanced generative capabilities but are constrained by static training corpora, producing hallucinated or outdated outputs when queried about recent developments. Retrieval-Augmented Generation (RAG) mitigates this by conditioning generation on externally retrieved evidence; however, baseline vector-only retrieval often suffers from contextual imprecision.

This report presents the implementation progress of TechPulse, a cloud-native Hybrid RAG system designed to synthesize emerging technology intelligence under operational latency and cost constraints. The system ingests data from five heterogeneous sources—ArXiv, Hacker News, DEV.to, GitHub Trending, and curated RSS/Atom feeds—with deterministic preprocessing, token-based chunking, and semantic embedding via the MiniLM sentence-transformer. Embeddings are indexed in PostgreSQL with the pgvector extension, supporting integrated relational filtering and vector similarity search.

The retrieval pipeline operates as a three-stage process: metadata-constrained filtering, cosine similarity search, and lightweight reranking incorporating semantic similarity, lexical overlap, and exponential recency weighting. An automatic LLM backend fallback chain ensures generation availability across deployment environments, with HuggingFace Inference API as the primary backend under AWS Academy Learner Lab constraints and Amazon Bedrock available under the AWS Free Tier setup. The system incorporates dual-criteria drift detection (Shewhart 3σ control charts and relative-drop thresholds), three-layer hallucination verification, and comprehensive health monitoring. An evaluation framework comparing baseline and hybrid retrieval has been designed and implemented using RAGAS metrics and grid search over reranking weights; full execution is planned for Phase B (March–April 2026).

This report documents the system architecture, pipeline design, implementation decisions, deviations from the original proposal, current progress, and remaining work.

Keywords: Retrieval-Augmented Generation; Hybrid Information Retrieval; Vector Databases; Cloud-Native Architecture; MLOps; Semantic Embedding; Emerging Technology Intelligence.

ACKNOWLEDGMENTS

The authors express sincere gratitude to Dr. Chantri Polprasert, the course instructor, for providing guidance on the system design and evaluation methodology throughout this project. We also thank the teaching assistants, Mr. Swaraj Bhanja and Mr. Suryansh Srivastava, for their constructive feedback during development milestones.

This project was developed and deployed in two AWS environments: the AWS Academy Learner Lab provided by the Asian Institute of Technology (AIT), and a personal AWS Free Tier account. We acknowledge AWS Academy for providing cloud computing resources that enabled the deployment and evaluation of the TechPulse system across both environments.

CONTENTS

ABSTRACT **i**

ACKNOWLEDGMENTS **ii**

LIST OF TABLES **vi**

LIST OF FIGURES **vii**

LIST OF ABBREVIATIONS **x**

1 Introduction **1**

1.1 Problem Description and Motivation 1
1.2 Research Questions 2
1.3 Objectives of the Study 2
1.4 Significance of the Study 3
1.5 Alignment Between Research Questions and Evaluation 3
1.6 Report Structure 4

2 Background and Related Work **5**

2.1 Retrieval-Augmented Generation 5
2.2 Embedding Models and Vector Search 5
2.3 Hybrid Retrieval and Reranking 5
2.4 RAG Evaluation Methodologies 6
2.5 Data and Concept Drift in Retrieval Systems 6
2.6 Hallucination Detection and Faithfulness Verification 7
2.7 MLOps and Production Machine Learning Systems 7
2.8 Research Gap 7
2.9 Chapter Summary 8

3 Methodology **9**

3.1 Overview of Technical Approach 9
3.2 Why This Approach is Suitable 9
3.3 Technical Contributions and Design Decisions 10
3.4 Baseline Retrieval Configuration 10
3.5 Implemented Hybrid Retrieval Configuration 11
3.6 Hallucination Verification Strategy 12
3.7 Optimization Objective 13
3.8 Tool Selection and Implementation Stack 13
3.9 System Context Overview 14

4 System Architecture and Pipeline Design **16**

4.1 Architectural Overview 16

4.2 Alignment with the AWS Well-Architected Framework 16
4.3 Data Sources 17
4.4 Data Engineering Pipeline 18
4.4.1 Ingestion and Decoupling 19
4.4.2 Document State Lifecycle 19
4.4.3 Preprocessing and Chunking 20
4.5 Embedding and Indexing 21
4.6 Storage Architecture (Medallion Layout) 21
4.7 Database Schema 21
4.8 Prompt Construction and Grounded Generation 22
4.9 LLM Backend Fallback Chain 23
4.10 Deployment, Monitoring, and MLOps 24
4.10.1 LLM Generation Configuration 24
4.10.2 Drift Detection and Remediation 25
4.10.3 Health Monitoring 26
4.10.4 CI/CD and Deployment Automation 26
4.11 AWS Cloud Deployment Architecture 27
4.12 User Interface 27

5 Experimental Plan and Evaluation Metrics 28
5.1 Evaluation Objective 28
5.2 Experimental Configurations 28
5.3 Evaluation Dataset 28
5.4 Evaluation Metrics 29
5.4.1 Context Precision 29
5.4.2 Faithfulness and Hallucination Verification 29
5.4.3 Latency 30
5.4.4 Cost Per Query 30
5.4.5 Reranking Weight Optimization 30
5.5 Planned Analysis and Discussion Framework 31
5.6 Planned Result Visualizations 31
5.6.1 Baseline vs. Hybrid Retrieval Comparison 31
5.6.2 Grid Search: Precision–Latency Pareto Frontier 32
5.6.3 Per-Source Retrieval Distribution 33
5.6.4 Latency Breakdown by Pipeline Stage 33
5.6.5 Evaluation Summary Dashboard 34

6 Implementation Progress and Deviations 35
6.1 Overall Implementation Status 35
6.2 Deviations from the Original Proposal 37

6.3 Expanded Data Source Coverage 39
6.4 Enhanced Preprocessing Pipeline 39
6.5 Dual-Environment Deployment: Learner Lab and AWS Free Tier 39
6.5.1 AWS Academy Learner Lab 39
6.5.2 AWS Free Tier 40
6.6 Test Coverage 40
6.7 Custom CloudWatch Metrics 41
6.8 Engineering Observations 41
6.9 System Demonstration 42
6.9.1 Frontend: Hybrid Retrieval Query with Citation-Grounded Response 42
6.9.2 Frontend: Baseline vs. Hybrid Comparison 43
6.9.3 Unit Test Execution 44
6.9.4 Docker Compose: Running Services 45
6.9.5 CI/CD Pipeline: GitHub Actions 46
6.9.6 Database: Populated Corpus from Five Sources 47

7 Remaining Work and Limitations 50
7.1 Remaining Work 50
7.1.1 Phase A: System Validation and Corpus Building (Mar W3 – Mar W4) 50
7.1.2 Phase B: Evaluation and Experimentation (Mar W4 – Apr W1) . 51
7.1.3 Phase C: Analysis and Monitoring Validation (Apr W1 – Apr W2) 51
7.1.4 Phase D: Final Report and Submission (Apr W2) 51
7.2 Known Limitations 52
7.3 Project Timeline 53
7.4 Team Contributions 53
7.5 Future Extensions 54

REFERENCES 56

A Source Code Availability 59

B Repository Structure 61

C System Demonstration Screenshots 63

LIST OF TABLES

1.1 Mapping of Research Questions to Evaluation Metrics and Methodology. 4

2.1 Comparison of Related Work and Positioning of TechPulse. 8

3.1 Implemented Tools, Services, and Justification. 14

4.1 TechPulse Alignment with the AWS Well-Architected Framework Six Pillars. 17
4.2 Implemented Data Sources and Roles. 18
4.3 Document State Transition Model. 19
4.4 Storage Layers and Responsibilities (Medallion Architecture). 21
4.5 Implemented LLM Backend Fallback Hierarchy. 23
4.6 Implemented LLM Generation Parameter Configuration. 25

5.1 Experimental Configuration Comparison. 28
5.2 Summary of Evaluation Metrics and Targets. 30
5.3 Planned Evaluation Summary Dashboard: Baseline vs. Hybrid with Target Thresholds. 34

6.1 Component Implementation Status Summary. 36
6.2 Deviations from the Original Proposal and Rationale. 38
6.3 Unit Test Distribution. 41

7.1 Remaining Work: Phase Overview. 50
7.2 Team Member Contribution Summary. 54

LIST OF FIGURES

3.1 Comparison of Baseline Vector-Only Retrieval (left) and Implemented Hybrid Retrieval Pipeline (right). The hybrid pipeline adds a metadata filtering stage to reduce the candidate pool and a reranking-lite stage for multi-factor relevance scoring prior to final candidate selection. 12

3.2 System Context Diagram for TechPulse. Solid arrows represent primary data and query flows; dashed arrows represent monitoring and evaluation signals. The system ingests from five external data sources and serves technology professionals via a query interface, while exposing evaluation and monitoring interfaces. 15

4.1 Six-Layer Conceptual Architecture of TechPulse. Right-hand solid arrows show the upward ingestion and generation data flow (raw documents → embeddings → retrieval → synthesis → response). The left-hand dashed arrow shows the downward query path from the user interface to the retrieval layer. Each layer has well-defined responsibilities and interfaces. 16

4.2 Implemented End-to-End Data Engineering and Serving Pipeline for TechPulse. Five heterogeneous sources feed into the ingestion pipeline. Solid arrows indicate data and control flow; dashed arrows indicate monitoring and observability signals. 18

4.3 Document State Machine. Each document progresses through four deterministic states. Transition labels show the triggering process (top) and the primary storage destination (bottom). SHA-256 deduplication at ingestion prevents duplicate entries entering the RAW state. 20

4.4 Entity–Relationship Diagram of the TechPulse PostgreSQL Schema. The `documents` table stores ingested records; the `chunks` table stores preprocessed segments with their 384-dimensional pgvector embeddings (HNSW-indexed); and the `drift_baselines` table persists drift monitoring history. The `content_hash` unique constraint on `documents` enforces idempotent ingestion. 22

4.5 Implemented MLOps lifecycle loop: continuous ingestion, preprocessing, indexing, serving, evaluation, and monitoring, with CI/CD automation and a budget governance guard. 24
4.6 AWS Cloud Deployment Architecture (Learner Lab configuration). Lambda functions, RDS, SQS, and S3 reside within the default VPC. EventBridge schedules ingestion; SQS decouples ingestion from embedding; API Gateway exposes the RAG API to the React frontend. Dashed arrows indicate monitoring and CI/CD flows. The Free Tier configuration replaces the default VPC with a custom VPC, uses custom IAM roles, and enables Amazon Bedrock as the primary LLM backend. 27

5.1 Evaluation framework: four measurement dimensions, each with a target threshold and the research question it addresses. Arrows point from measurement method to the metric it produces. A Wilcoxon signed-rank test will assess the statistical significance of the precision difference between baseline and hybrid configurations. 29
5.2 Planned comparison of baseline vs. hybrid retrieval across four evaluation dimensions. Bars represent mean scores across 25 evaluation queries. Latency is normalized to the 2-second target (value of 1.0 = target met). Actual values will be populated after evaluation execution. . 32
5.3 Planned grid search results: reranking similarity weight (α) vs. mean context precision (left axis, solid) and p95 latency (right axis, dashed). The horizontal dashed line marks the 2-second latency constraint. The optimal α maximizes precision without exceeding the latency threshold. Constraint: $\beta = \gamma = (1-\alpha)/2$. 32
5.4 Planned per-source retrieval distribution for baseline vs. hybrid modes. Stacked bars show the proportion of top-k retrieved documents originating from each of the five data sources, averaged across 25 evaluation queries. Greater diversity in the hybrid configuration would support the retrieval hypothesis. 33
5.5 Planned latency decomposition by pipeline stage for baseline and hybrid retrieval. The baseline omits metadata filtering and reranking stages. The vertical dashed line marks the 2-second p95 latency target. Actual values will be populated from per-query timing measurements. 33

6.1 TechPulse frontend: hybrid retrieval query with citation-grounded response. The response displays inline source citations and a ranked source list with relevance scores from all five data sources. 43

6.2 TechPulse frontend: baseline (vector-only) retrieval query on the same question as Figure 6.1, enabling visual comparison of response quality and source diversity. 44
6.3 Pytest execution: 49 unit tests passing across 5 test modules (`test_preprocessing`, `test_ingestion`, `test_retriever`, `test_rag`, `test_api`). 45
6.4 Docker Compose deployment: six services running locally — PostgreSQL+pgvector, pgAdmin, LocalStack, FastAPI API, React frontend, and the ingestion scheduler. 46
6.5 GitHub Actions CI/CD pipeline: four-stage workflow (lint & test, Docker build, SAM build, deploy) executing successfully on the `main` branch. . 47
6.6 pgAdmin: the `documents` table populated with records from all five data sources (ArXiv, Hacker News, DEV.to, GitHub Trending, RSS), demonstrating successful multi-source ingestion. 48

7.1 Project timeline (Gantt chart). Solid bars indicate completed phases; outlined bars indicate remaining work organized into four phases (A–D). The dashed vertical line marks the progress report submission. All remaining work is scheduled to complete by second week of April 2026. 53

C.1 Health check endpoint (`/health`): JSON response confirming database connectivity, document/chunk counts, S3 reachability, SQS reachability, and LLM backend status. 63
C.2 Drift detection endpoint (`/drift`): JSON response showing probe query mean similarity, standard deviation, alert status, and Shewhart 3σ control analysis. 64
C.3 RAG query endpoint (`POST /ask`): full JSON response showing the generated answer, retrieved source documents with titles, URLs, similarity scores, and retrieval mode metadata. 64

LIST OF ABBREVIATIONS

AIT	Asian Institute of Technology
ANN	Approximate Nearest Neighbor
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
DLQ	Dead Letter Queue
HF	HuggingFace
HNSW	Hierarchical Navigable Small World
IaC	Infrastructure as Code
LCL	Lower Control Limit
LLM	Large Language Model
MLOps	Machine Learning Operations
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RAGAS	Retrieval-Augmented Generation Assessment
RDS	Relational Database Service
RSS	Really Simple Syndication
S3	Simple Storage Service
SAM	Serverless Application Model
SHA-256	Secure Hash Algorithm (256-bit)
SNS	Simple Notification Service
SPA	Single-Page Application
SQL	Structured Query Language
SQS	Simple Queue Service
VPC	Virtual Private Cloud

CHAPTER 1

INTRODUCTION

1.1 Problem Description and Motivation

Motivating scenario. A machine learning engineer at a technology company needs to stay current with advances in transformer efficiency. She must manually monitor ArXiv preprints, scan Hacker News threads, and cross-reference multiple sources—a process that is time-consuming, fragmented, and prone to missing recent developments. An intelligent assistant capable of synthesizing both scholarly and community-sourced knowledge into a single, citation-grounded response would significantly accelerate her workflow. TechPulse is designed precisely for this use case.

The exponential growth of emerging technologies—including artificial intelligence, machine learning, distributed systems, and advanced computing—has produced an unprecedented volume of technical knowledge distributed across heterogeneous platforms. Scholarly repositories such as ArXiv provide structured, peer-reviewed or preprint research outputs contributing to theoretical and methodological advancement. Community-driven platforms and industry discussion forums such as Hacker News reflect near-real-time practitioner experiences and emerging tool ecosystems. However, the fragmentation of these knowledge regimes creates an information landscape that is increasingly difficult for technology professionals to synthesize effectively and in a timely manner.

Purely generative large language model (LLM) assistants are insufficient for emerging technology intelligence tasks due to their static knowledge cutoff, susceptibility to hallucinated responses, lack of verifiable evidence grounding, and inability to integrate with continuously updating data streams (Ji et al., 2023). Although Retrieval-Augmented Generation (RAG) architectures partially address these limitations by incorporating external context, baseline vector-only retrieval approaches may still retrieve semantically adjacent but contextually irrelevant documents. Such systems often fail to incorporate temporal relevance signals, structured metadata constraints, or recency-aware ranking mechanisms, which are essential for accurately capturing rapidly evolving technology trends.

Moreover, many existing RAG research prototypes focus primarily on retrieval accuracy without addressing production-readiness: deterministic pipeline design, idempotent ingestion, monitoring and observability, deployment automation, and cloud cost governance are frequently neglected. This limits their applicability in real-world operational

environments.

The central engineering and research challenge addressed in this project is therefore:

> ***How can a scalable, continuously updating hybrid retrieval system be designed and deployed to synthesize emerging technology trends from heterogeneous scholarly and industry sources while satisfying latency and cost constraints?***

This project is motivated by three converging factors. First, technology professionals increasingly require timely and citation-grounded intelligence that synthesizes insights from both scholarly research and rapidly evolving industry discussions. Second, mature serverless cloud infrastructures now enable cost-effective deployment of production-grade data and machine learning pipelines. Third, a gap remains in the Retrieval-Augmented Generation (RAG) literature between retrieval-focused research prototypes and operationally robust systems capable of continuous ingestion, evaluation, and deployment in real-world environments.

1.2 Research Questions

To address the central problem, this study investigates the following research questions:

1. **Data Engineering.** How can a cloud-native pipeline be designed to reliably ingest and preprocess heterogeneous scholarly and near-real-time industry data in a deterministic and idempotent manner?
2. **Retrieval Optimization.** Does hybrid retrieval—combining metadata filtering, vector similarity search, and lightweight reranking—improve retrieval precision and grounding faithfulness compared with baseline vector-only retrieval?
3. **Operational Trade-offs.** What trade-offs arise between retrieval precision, p95 latency, and cost per query under realistic workload conditions?
4. **MLOps Integration.** How can deployment, monitoring, version control, and cost governance be integrated to ensure the reliability and maintainability of a production-grade RAG system?

1.3 Objectives of the Study

The primary objective is to design, implement, and evaluate a hybrid Retrieval-Augmented Generation system capable of synthesizing real-time emerging technology intelligence within a scalable, cost-aware cloud-native architecture. The specific objectives are:

1. To construct a cloud-native data engineering pipeline for batch and near-real-time

ingestion from five heterogeneous data sources.

2. To implement and compare baseline vector-only retrieval against hybrid retrieval with metadata filtering and reranking-lite optimization.
3. To evaluate system performance across context precision, faithfulness, p95 latency, and cost per query.
4. To deploy and monitor the system using serverless AWS infrastructure consistent with MLOps best practices (Sculley et al., 2015; Breck et al., 2017).

1.4 Significance of the Study

This project contributes at the intersection of data engineering, natural language processing, and MLOps. From a technical perspective, it demonstrates how retrieval optimization can be integrated within a production-grade cloud architecture without relying on computationally expensive cross-encoder rerankers or specialized vector database infrastructure. The system provides a practical framework for grounded intelligence synthesis across academic and industry domains while maintaining architectural simplicity suitable for scalable deployment. From a practical perspective, the resulting system offers a blueprint for citation-supported technology intelligence synthesis that is operationally maintainable, cost-governed, and deployable within resource-constrained environments such as the AWS Academy Learner Lab and the AWS Free Tier.

1.5 Alignment Between Research Questions and Evaluation

Table 1.1 provides an explicit mapping from each research question to the corresponding evaluation metric and experimental configuration used to answer it.

Table 1.1. Mapping of Research Questions to Evaluation Metrics and Methodology.

RQ	Question (abbreviated)	Primary Metric	Evaluation Approach
1	Cloud-native deterministic ingestion	Pipeline idempotency; SHA-256 deduplication pass rate	Unit tests (18 ingestion tests); repeated pipeline runs
2	Hybrid vs. baseline retrieval precision	Context precision (RAGAS); faithfulness ≥ 0.80; citation grounding > 0.80	Paired comparison over 25 queries; Wilcoxon signed-rank test
3	Latency and cost trade-offs	p95 latency $<$ 2s; cost $<$ USD 15/month	Per-query timing; token usage profiling; grid search Pareto frontier
4	MLOps integration	Drift detection coverage; CI/CD pass rate; health-check uptime	Drift simulation runs; CloudWatch alarm verification

1.6 Report Structure

Chapter 1 introduces the problem, research questions, objectives, and their mapping to evaluation metrics. Chapter 2 reviews relevant literature on Retrieval-Augmented Generation, hybrid retrieval, vector databases, and MLOps. Chapter 3 presents the methodology, hypotheses, and technical approach. Chapter 4 describes the implemented system architecture. Chapter 5 presents the evaluation framework and planned analyses. Chapter 6 documents implementation status, deviations, and engineering observations. Chapter 7 summarizes remaining work, known limitations, and the completion timeline.

Chapter Summary. This chapter established the problem context, four research questions, and the core motivation for a hybrid RAG system targeting real-time technology intelligence synthesis. The following chapter reviews the technical and scientific foundations underpinning the design decisions made in this project.

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. (2020) as a framework for augmenting sequence-to-sequence generation with a dense retrieval component, enabling the model to access external documents at inference time. This approach significantly improves performance on knowledge-intensive tasks by conditioning generation on retrieved evidence rather than relying solely on parametric memory. Despite their scale, LLMs remain prone to hallucination when responding to domain-specific or time-sensitive queries (Ji et al., 2023), making retrieval-based augmentation a practically important technique.

Recent surveys (Gao et al., 2023; Zhang et al., 2024) categorize RAG systems into four architectural types: (1) single-stage dense retrieval; (2) hybrid lexical–semantic retrieval; (3) multi-stage retrieval with reranking; and (4) iterative or self-reflective retrieval. While baseline dense retrieval improves factual grounding, empirical benchmarks such as BEIR (Thakur et al., 2021) demonstrate that semantic similarity alone is insufficient for high-precision retrieval in dynamic or heterogeneous corpora. Foundational retrieval-augmented systems such as Atlas (Izacard et al., 2022) and RETRO (Borgeaud et al., 2022) demonstrate retrieval augmentation at scale but require computationally intensive pretraining pipelines unsuited to cost-constrained production environments.

2.2 Embedding Models and Vector Search

Dense retrieval relies on embedding models that transform text into high-dimensional vector representations. Sentence-transformer architectures such as MiniLM (Wang et al., 2020) provide computationally efficient embeddings with favorable trade-offs between semantic expressiveness and inference latency. PostgreSQL with the pgvector extension enables cosine similarity search alongside SQL-based metadata filtering within a single relational storage layer, reducing architectural complexity compared to dedicated vector databases such as FAISS or Milvus. This integrated approach is increasingly favored for production RAG deployments (Gao et al., 2023).

2.3 Hybrid Retrieval and Reranking

Hybrid retrieval combines lexical matching (e.g., BM25 or keyword filtering) with semantic similarity search to improve retrieval precision. Thakur et al. (2021) demonstrate

that hybrid approaches consistently outperform purely dense retrieval across heterogeneous benchmarks. Reranking further refines candidate relevance using additional contextual signals. Cross-encoder rerankers achieve high accuracy but incur substantial computational overhead (Nogueira & Cho, 2019); lightweight multi-factor reranking—combining similarity score, recency weighting, and keyword overlap—offers comparable grounding improvements at lower latency (Gao et al., 2023). In dynamic knowledge domains, recency-aware retrieval is particularly important to avoid outdated context injection.

2.4 RAG Evaluation Methodologies

Evaluating RAG systems requires assessing both retrieval quality and generation faithfulness simultaneously. Es et al. (2023) introduced RAGAS, a reference-free evaluation framework that measures faithfulness, answer relevance, context precision, and context recall using automated LLM-based scoring, enabling scalable evaluation without large human annotation pools. Complementary work by Saad-Falcon et al. (2023) proposed ARES, which uses a small set of human-annotated examples to train lightweight judges capable of scoring RAG systems across multiple quality dimensions. These frameworks demonstrate that even compact evaluation sets—in the range of 20 to 50 annotated examples—can yield statistically meaningful comparisons between retrieval configurations when evaluation dimensions are clearly scoped and metrics are carefully defined (Es et al., 2023). This finding provides empirical support for the evaluation scale adopted in TechPulse.

2.5 Data and Concept Drift in Retrieval Systems

In dynamic knowledge domains, retrieval system performance can degrade over time due to data drift—a shift in the statistical properties of incoming documents relative to the indexed corpus—or concept drift, where the semantic meaning of user queries evolves independently of corpus updates (Gama et al., 2014). Lu et al. (2018) provide a comprehensive taxonomy of drift detection methods, categorizing approaches into statistical process control, windowed distribution comparison, and ensemble-based disagreement detection. In retrieval-augmented systems specifically, drift can manifest as declining context precision when newly ingested documents diverge from the embedding space learned during the original indexing phase. Addressing drift requires both monitoring mechanisms that can detect distributional shifts in retrieval scores over time, and remediation actions such as corpus re-indexing, embedding model refresh, or query routing adjustments (Bayram et al., 2022). TechPulse incorporates a dual-criteria embedding distribution monitoring strategy informed by these findings.

2.6 Hallucination Detection and Faithfulness Verification

Despite retrieval augmentation, generative models can still produce hallucinated content by inferring beyond the supplied context, over-generalizing retrieved passages, or introducing spurious cross-document associations (Ji et al., 2023). Manakul et al. (2023) proposed SelfCheckGPT, a consistency-based approach that detects hallucinations by sampling multiple outputs and measuring inter-sample consistency—a technique applicable without access to ground-truth references. Min et al. (2023) proposed FActScoring, which decomposes generated responses into atomic claims and individually verifies each claim against a reference corpus, offering fine-grained faithfulness assessment. At a system design level, Shi et al. (2023) demonstrate that irrelevant context injected alongside relevant passages can distract generative models and induce hallucinated responses even when correct information is present—motivating strict top-k selection and high-precision retrieval as the primary anti-hallucination mechanism in TechPulse.

2.7 MLOps and Production Machine Learning Systems

Sculley et al. (2015) identify hidden technical debt as a central challenge in ML systems, emphasizing monitoring, reproducibility, and lifecycle management. MLOps extends DevOps principles to ML workflows, incorporating continuous integration, deployment automation, model versioning, and observability (Breck et al., 2017). Cloud-native, serverless architectures enable scalable and maintainable deployment with cost governance (Zhang et al., 2020; AWS, 2023). While RAG research frequently focuses on retrieval effectiveness, production-readiness—including pipeline idempotency, schema versioning, and budget-aware inference—remains underexplored.

2.8 Research Gap

The reviewed literature establishes six key insights: (1) RAG improves factual grounding but baseline dense retrieval may lack precision; (2) hybrid retrieval outperforms vector-only search in heterogeneous settings; (3) lightweight reranking can improve relevance without prohibitive overhead; (4) production ML systems require robust MLOps practices beyond model accuracy; (5) even compact evaluation sets of 20–50 annotated examples can yield statistically meaningful RAG comparisons when evaluation dimensions are clearly scoped; and (6) hallucination in RAG systems can persist despite retrieval augmentation, necessitating explicit faithfulness verification mechanisms.

Critically, existing hybrid RAG systems either address a subset of these concerns or are evaluated in static, single-source corpora that do not reflect the temporal dynamics of multi-source technology intelligence. Atlas (Izacard et al., 2022) and RETRO (Borgeaud et al., 2022) operate at scale but require pretraining pipelines and do not address operational cost governance or retrieval-quality drift. Production systems such as Perplexity

AI and Bing Copilot incorporate real-time retrieval but do not expose evaluation methodology or reranking designs. Academic RAG prototypes frequently omit monitoring, idempotent ingestion, and budget constraints that are essential for sustained deployment. No existing published system integrates all six dimensions—hybrid retrieval, lightweight reranking, real-time multi-source ingestion, cost governance, MLOps monitoring, and explicit hallucination verification—into a unified, student-deployable, cloud-native architecture targeting dynamic emerging technology intelligence. TechPulse addresses this combined gap.

Table 2.1 summarizes representative related work and positions TechPulse relative to prior systems.

Table 2.1. Comparison of Related Work and Positioning of TechPulse.

System	**Retrieval**	**Rerank**	**Real-Time**	**Cost Gov.**	**MLOps**	**Drift Det.**	**Halluc. Verif.**
RAG (Lewis et al., 2020)	Dense	No	No	No	No	No	No
Atlas (Izacard et al., 2022)	Dense	No	No	No	No	No	No
RETRO (Borgeaud et al., 2022)	Dense	No	No	No	No	No	No
Typical RAG Prototype	Dense	Optional	Rare	No	Minimal	Rare	Rare
TechPulse (ours)	**Hybrid**	**Yes**	**Yes**	**Yes**	**Full**[†]	**Dual**	**3-Layer**

[†]Full MLOps is implemented and under cloud validation; evaluation results are pending Phase B execution.

2.9 Chapter Summary

This chapter reviewed the evolution of RAG architectures, embedding-based retrieval, hybrid retrieval optimization, MLOps principles, RAG evaluation methodologies, drift detection strategies, and hallucination verification approaches. The literature confirms that hybrid retrieval with lightweight reranking and production-grade MLOps integration represents an underexplored but practically important direction, and that compact but well-scoped evaluation sets can yield statistically meaningful RAG comparisons. Importantly, no existing system integrates all six identified dimensions into a unified, cost-aware, cloud-native architecture for dynamic technology intelligence—the gap that TechPulse directly addresses. The following chapter presents the methodology and technical design decisions that operationalise this positioning.

CHAPTER 3

METHODOLOGY

3.1 Overview of Technical Approach

TechPulse has been implemented as a cloud-native Hybrid Retrieval-Augmented Generation system designed to synthesize emerging technology intelligence from heterogeneous sources. The technical approach is organized around five interconnected components: (1) continuous heterogeneous data ingestion from five sources; (2) deterministic preprocessing and semantic embedding; (3) hybrid multi-stage retrieval; (4) context-bound grounded generation with automatic LLM fallback; and (5) production-grade MLOps deployment and monitoring.

The core hypotheses under evaluation are stated formally as:

- $\mathbf{H_0}$: There is no statistically significant difference in mean context precision between hybrid and baseline (vector-only) retrieval, evaluated over 25 matched queries under identical corpus and embedding conditions.
- $\mathbf{H_1}$: Hybrid retrieval achieves statistically higher mean context precision than baseline retrieval (one-sided Wilcoxon signed-rank test, $\alpha = 0.05$).

A secondary performance hypothesis requires that the precision improvement does not exceed the p95 latency bound ($\tau = 2.0$ seconds for the retrieval stage) or the monthly cost ceiling (USD 15 at an assumed workload of 1,500 queries/month, i.e., USD 0.01 per query). These hypotheses are tested through a controlled experiment comparing both configurations under identical corpus and embedding conditions, with full results reported in the final submission.

3.2 Why This Approach is Suitable

Three considerations motivate the hybrid RAG approach for this specific problem domain.

Domain characteristics. Emerging technology knowledge is temporally dynamic and distributed across heterogeneous sources. Vector-only retrieval treats all documents equally regardless of recency or source authority; hybrid retrieval with recency-aware reranking explicitly addresses temporal drift, which is critical in rapidly evolving AI/ML domains where a paper from 18 months ago may already be superseded.

Operational constraints. The system must operate within cost and latency bounds suitable for a research prototype deployed across two AWS environments: the AWS Academy Learner Lab and the AWS Free Tier. The lightweight linear reranking function (Equation 3.2) adds $O(k)$ computation over the top-N filtered candidates—negligible compared to the $O(Nd)$ baseline search—and avoids the latency penalties of cross-encoder rerankers, which typically require a

forward pass through a BERT-class model per candidate.

Production feasibility. The architecture uses AWS-managed services—Lambda, RDS PostgreSQL, API Gateway, EventBridge—eliminating the need to manage dedicated compute clusters or self-hosted vector databases. This enables a student team to deploy, iterate, and monitor a production-grade system within the course timeline and a prototype budget envelope.

3.3 Technical Contributions and Design Decisions

The engineering contribution of TechPulse lies not in any single component but in their integration into a coherent, cost-governed MLOps system. Specifically, four design decisions represent well-motivated engineering choices that distinguish this system from typical RAG prototypes.

Recency-aware hybrid reranking for dynamic knowledge domains. The weighted linear reranking function (Equation 3.2) fuses semantic similarity, lexical keyword overlap, and exponential temporal decay into a single interpretable score. Unlike cross-encoder rerankers that are domain-agnostic and computationally expensive, this formulation is specifically designed for domains where content freshness is a first-class retrieval signal. The exponential decay parameter λ provides a principled, tunable mechanism for controlling how rapidly older documents lose retrieval priority.

Unified vector–relational retrieval without dedicated infrastructure. By extending PostgreSQL with pgvector, TechPulse achieves hybrid retrieval (structured SQL metadata filtering + cosine vector search) within a single managed database, eliminating the operational complexity of running a separate vector store. This design choice reduces architectural surface area, simplifies deployment automation, and enables cost-governed scaling through standard RDS instances.

Idempotent ingestion pipeline with SHA-256 content-addressed deduplication. The SHA-256 hashing strategy applied to canonicalized document fields ensures that the pipeline can be re-run safely at any time without creating duplicate records. This is an often-overlooked but practically important data engineering innovation—it enables safe experiment replay, crash recovery, and corpus versioning without requiring a separate deduplication service.

Budget-aware inference with programmatic halt. The integration of AWS Cost Explorer queries into the inference path—where invocations are halted programmatically when monthly cost thresholds are approached—represents an operational optimization strategy not typically addressed in RAG research prototypes. This transforms cost from a passive monitoring concern into an active constraint enforced at runtime, consistent with production-grade MLOps principles.

3.4 Baseline Retrieval Configuration

The baseline configuration performs dense cosine similarity search across the full indexed corpus. Given a query embedding $\mathbf{e}_q$ and document embeddings $\mathbf{e}_i$, cosine similarity is computed as:

$$\mathrm{Sim}(q,i) = \frac{\mathbf{e}_q \cdot \mathbf{e}_i}{\|\mathbf{e}_q\|\|\mathbf{e}_i\|} \tag{3.1}$$

Top-N documents are selected purely on similarity ranking with a minimum similarity threshold of 0.15 to filter retrieval noise. This low threshold is intentionally permissive in the baseline to avoid artificially handicapping it; borderline documents that would be excluded by a stricter threshold are instead ranked low by the subsequent reranking stage in the hybrid configuration. Under exact search, computational complexity scales as $O(Nd)$, where N is the number of indexed chunks and $d = 384$ (the MiniLM embedding dimension). This configuration serves as the experimental control for retrieval performance comparison.

3.5 Implemented Hybrid Retrieval Configuration

The hybrid configuration introduces three sequential refinement stages applied before generation.

Stage 1 — Metadata Filtering. Structured SQL constraints reduce the candidate space based on document recency (a 180-day sliding window), source type, and document state. Only documents in the `INDEXED` state are considered. The 180-day window was chosen to balance corpus recency with sufficient document volume: at the expected ingestion rates across five sources, the window is projected to yield 500–2,000 indexed chunks, providing a meaningful retrieval pool while excluding content more than six months old, which is considered stale for fast-moving AI/ML topics. This pre-filtering stage reduces retrieval noise prior to semantic ranking.

Stage 2 — Vector Similarity Search. Cosine similarity (Equation 3.1) is applied over the filtered candidate subset using the pgvector `<=>` operator. To provide sufficient candidates for reranking, $4 \times$ top_k results are retrieved at this stage. The $4\times$ multiplier follows the oversampling convention used in cross-encoder reranking pipelines (Nogueira & Cho, 2019), giving the reranker enough diversity to meaningfully reorder results without expanding the candidate pool to the point where reranking overhead dominates latency.

Stage 3 — Reranking-Lite. A lightweight weighted scoring function refines candidate ordering:

$$\text{Score}_i = \alpha \cdot \text{Sim}_i + \beta \cdot \text{KeywordOverlap}_i + \gamma \cdot \text{RecencyWeight}_i \tag{3.2}$$

where $\alpha + \beta + \gamma = 1$. The implemented default weights are $\alpha = 0.6$, $\beta = 0.2$, $\gamma = 0.2$, configurable via environment variables (`RERANK_ALPHA`, `RERANK_BETA`, `RERANK_GAMMA`) to support grid search during evaluation.

Recency is modeled using exponential decay:

$$\text{RecencyWeight}_i = e^{-\lambda \Delta t_i} \tag{3.3}$$

where Δt_i is document age in days and $\lambda = 0.01$ (configurable via `RECENCY_LAMBDA`). At this decay rate, a document published 69 days ago retains $e^{-0.69} \approx 0.50$ of its recency weight (the "half-life" of relevance ≈ 69 days, or approximately 2.3 months), and a document at the 180-day filter boundary retains $e^{-1.8} \approx 0.17$. This profile reflects the observation that AI/ML content

has a practical relevance half-life of approximately 2–3 months; λ is configurable to enable sensitivity analysis during evaluation.

Figure 3.1 illustrates the conceptual difference between the baseline and hybrid retrieval pathways.

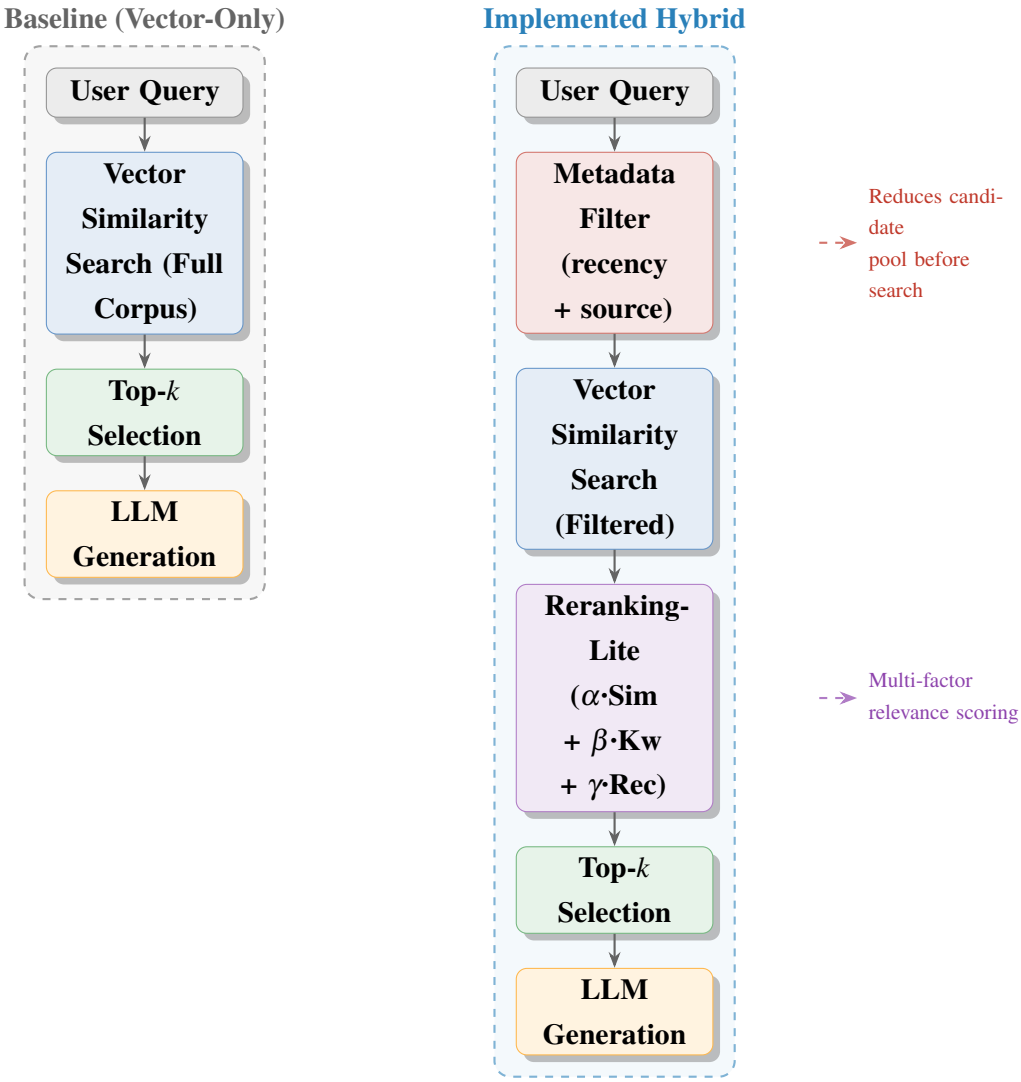

Figure 3.1. Comparison of Baseline Vector-Only Retrieval (left) and Implemented Hybrid Retrieval Pipeline (right). The hybrid pipeline adds a metadata filtering stage to reduce the candidate pool and a reranking-lite stage for multi-factor relevance scoring prior to final candidate selection.

3.6 Hallucination Verification Strategy

TechPulse applies a three-layer verification approach to detect and flag potentially hallucinated outputs. The prompt construction and template details are presented in Section 4.8.

1. **Prompt layer.** The system instruction enforces strict evidence grounding and abstention on insufficient context.

2. **Evaluation layer.** RAGAS faithfulness scoring decomposes each generated response

into atomic claims and verifies whether each claim is supported by at least one retrieved document.

3. **Output layer — Citation traceability.** At runtime, automated citation grounding verification analyzes each response sentence-by-sentence to compute a citation ratio—the fraction of sentences containing valid `[Source N]` references. Responses falling below the configurable citation grounding threshold are flagged as potentially hallucinated and replaced with an abstention message, with the original response preserved in metadata for review.

3.7 Optimization Objective

The retrieval task is framed as a constrained optimization problem:

$$\max \text{ Precision}(q) \quad \text{subject to} \quad \text{Latency} \leq \tau \tag{3.4}$$

where τ is the p95 latency target of 2 seconds. This formulation makes the precision–latency trade-off explicit and provides a principled basis for tuning reranking depth and metadata filter selectivity.

3.8 Tool Selection and Implementation Stack

Table 3.1 summarizes the implemented tools and their justification. The stack has been adapted from the original proposal to accommodate AWS Academy Learner Lab constraints, with additional configuration for the AWS Free Tier environment, as detailed in Chapter 6.

Table 3.1. Implemented Tools, Services, and Justification.

Component	Tool / Service	Justification
Ingestion orchestration	AWS Lambda + EventBridge	Serverless, scheduled/event-driven, no cluster management
Message queue	Amazon SQS + DLQ	Decouples ingestion from embedding; failure resilience
Raw storage	Amazon S3 (medallion layout)	Cost-effective, durable, prefix-based lineage tracking
Embedding model	MiniLM (sentence-transformers)	Low latency, 384-dim, serverless-compatible
Vector + relational store	RDS PostgreSQL + pgvector	Unified hybrid retrieval; compatible with both Learner Lab and Free Tier
Generative model	HuggingFace / Ollama (fallback) in Learner Lab; Bedrock (Claude Haiku) in Free Tier	Multi-backend with auto-cascade; Bedrock unavailable in Learner Lab but enabled in Free Tier
Frontend	React (S3 static hosting)	Lightweight, decoupled from backend
API routing	Amazon API Gateway (HTTP)	Managed, HTTPS, CORS support
CI/CD	GitHub Actions + SAM	Reproducible IaC deployments, automated testing
Monitoring	CloudWatch + SNS alarms	Observability, error alerts, health checks
Evaluation	RAGAS + grid search	Standardized RAG quality metrics + weight optimization

3.9 System Context Overview

Figure 3.2 presents a high-level context view of TechPulse, showing external actors, data flows, and system boundaries before the detailed architecture is described in Chapter 4.

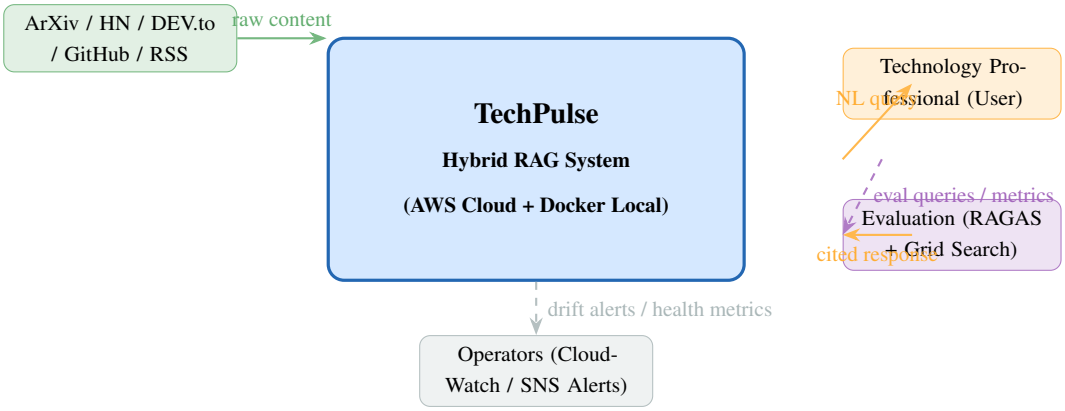

Figure 3.2. System Context Diagram for TechPulse. Solid arrows represent primary data and query flows; dashed arrows represent monitoring and evaluation signals. The system ingests from five external data sources and serves technology professionals via a query interface, while exposing evaluation and monitoring interfaces.

Chapter Summary. This chapter presented the methodology underpinning TechPulse: the formal hypotheses (H_0/H_1), the rationale for the hybrid approach, the design of the three-stage retrieval pipeline, hallucination verification strategy, parameter choices and their justifications, and the tool stack. The next chapter translates these methodological decisions into a concrete system architecture and data engineering pipeline design.

CHAPTER 4

SYSTEM ARCHITECTURE AND PIPELINE DESIGN

4.1 Architectural Overview

TechPulse has been implemented as a six-layer, cloud-native system deployed in two AWS environments: the AWS Academy Learner Lab and a personal AWS Free Tier account. Each layer is logically independent with well-defined interfaces, enabling modular development, testing, and future extension. The architecture was initially adapted from the original proposal to replace services unavailable in the Learner Lab (Aurora Serverless, Amazon Bedrock, custom VPCs, AWS Budgets) with compatible alternatives. The Free Tier setup restores access to several of these services, including Amazon Bedrock, while preserving the core hybrid retrieval and MLOps capabilities across both environments.

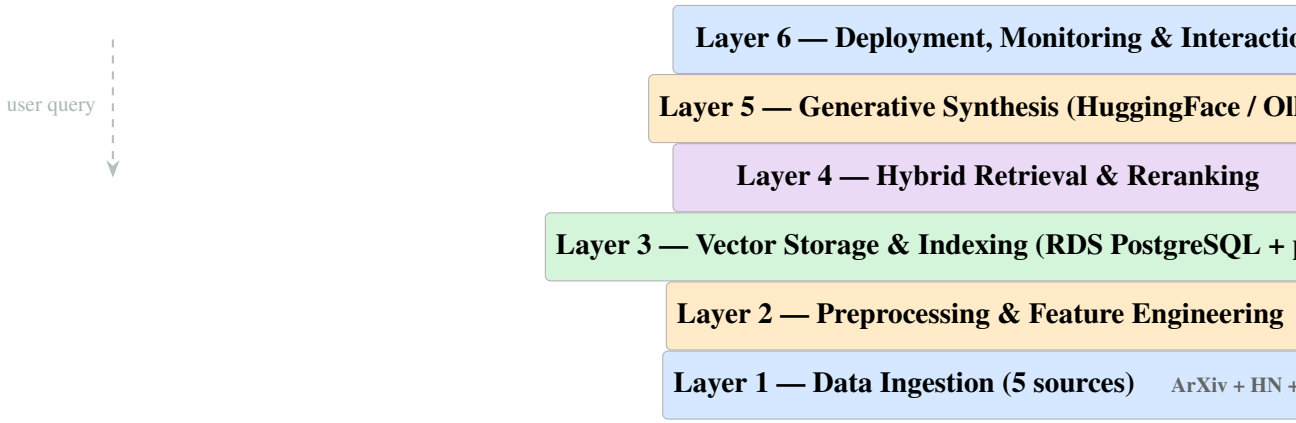

Figure 4.1. Six-Layer Conceptual Architecture of TechPulse. Right-hand solid arrows show the upward ingestion and generation data flow (raw documents → embeddings → retrieval → synthesis → response). The left-hand dashed arrow shows the downward query path from the user interface to the retrieval layer. Each layer has well-defined responsibilities and interfaces.

4.2 Alignment with the AWS Well-Architected Framework

The TechPulse architecture is designed in explicit alignment with the AWS Well-Architected Framework (AWS, 2023). Table 4.1 summarises how each pillar is addressed within the implemented system.

Table 4.1. TechPulse Alignment with the AWS Well-Architected Framework Six Pillars.

Pillar	TechPulse Implementation
Operational Excellence	Automated ingestion via EventBridge (6-hour cycle); CI/CD pipeline using GitHub Actions and SAM; CloudWatch logging, custom metrics, and health-check Lambda (5-minute interval)
Security	Controlled API access via Amazon API Gateway with CORS; rate limiting via slowapi; all services managed within AWS default VPC; no direct model invocation from the frontend
Reliability	Decoupled ingestion and embedding pipelines via SQS; Dead Letter Queue for failure recovery and replay with 14-day retention; bounded retry logic with exponential backoff for LLM invocations; automatic LLM backend fallback chain
Performance Efficiency	Serverless Lambda scaling; lightweight MiniLM embedding model (384-dim); metadata pre-filtering reduces vector search scope; HNSW index for approximate nearest-neighbor search
Cost Optimization	Serverless-only execution (pay-per-invocation); programmatic budget guard via Cost Explorer; HuggingFace free-tier inference as primary backend in Learner Lab; Amazon Bedrock available in Free Tier; db.t3.micro RDS instance
Sustainability	Event-driven processing avoids idle compute; SQS decoupling prevents over-provisioning; right-sized Lambda memory allocations (512MB–1024MB)

4.3 Data Sources

The original proposal specified two data sources (ArXiv and Hacker News). During implementation, the data source coverage was expanded to five heterogeneous sources to improve corpus diversity, topical breadth, and temporal coverage. Table 4.2 summarizes the implemented data sources.

Table 4.2. Implemented Data Sources and Roles.

Source	Type	Role	Frequency
ArXiv API	Batch	Scholarly grounding corpus (cs.AI, cs.LG, cs.CL, cs.IR, cs.SE)	Every 12 hours
Hacker News API	Near-real-time	Community-curated tech discussions	Every 30 minutes
DEV.to API	Near-real-time	Practitioner tutorials and industry articles (21 tags)	Every 6 hours
GitHub Trending	Batch	Open-source project intelligence (10 topic queries)	Every 6 hours
RSS/Atom Feeds	Batch	Curated technology journalism (10 feeds)	Every 6 hours

The combination provides complementary temporal and authority profiles: ArXiv contributes rigorously authored research with stable identifiers; Hacker News contributes practitioner signals and community-curated trending discussions; DEV.to provides developer-focused tutorials and implementation insights; GitHub Trending captures emerging open-source tools and frameworks; and curated RSS feeds from outlets including Ars Technica, TechCrunch, Wired, IEEE Spectrum, and MIT News provide technology journalism coverage.

4.4 Data Engineering Pipeline

The pipeline has been implemented around four design principles: determinism (identical input produces identical output), idempotency (re-running does not create duplicate records), observability (all state transitions are logged), and failure resilience (failures are isolated and recoverable).

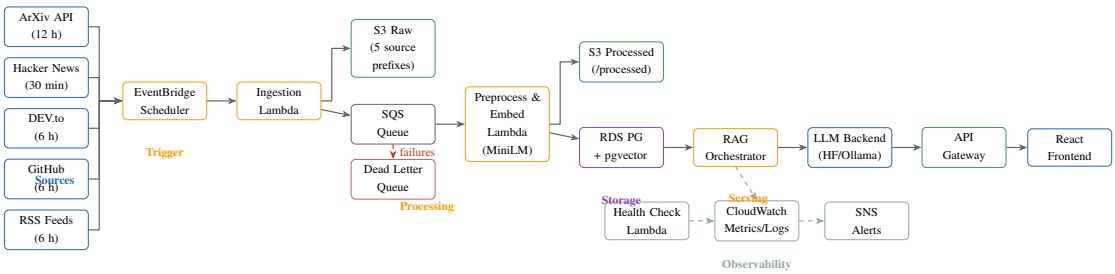

Figure 4.2. Implemented End-to-End Data Engineering and Serving Pipeline for TechPulse. Five heterogeneous sources feed into the ingestion pipeline. Solid arrows indicate data and control flow; dashed arrows indicate monitoring and observability signals.

4.4.1 Ingestion and Decoupling

Ingestion has been implemented through AWS Lambda functions orchestrated by Amazon EventBridge, supporting both scheduled and event-driven execution. Each of the five ingesters (ArXiv, Hacker News, DEV.to, GitHub, RSS) operates independently with source-specific data extraction logic. Raw artifacts are persisted in Amazon S3 under a structured prefix hierarchy (`/raw/arxiv/`, `/raw/hn/`, `/raw/devto/`, `/raw/github/`, `/raw/rss/`, `/processed/`) to preserve transformation lineage. The ingestion pipeline is decoupled from the embedding pipeline via Amazon SQS, providing backpressure control, failure isolation, and independent horizontal scaling. Messages that fail processing after three retry attempts are routed to a Dead Letter Queue with 14-day retention for controlled replay.

All five ingesters share a common architectural pattern: (1) fetch data from API or feed; (2) extract and normalize fields; (3) compute SHA-256 content hash over canonicalized fields; (4) insert into the `documents` table with `ON CONFLICT (content_hash) DO NOTHING` for idempotent deduplication; (5) write raw JSON to S3; and (6) enqueue document ID to SQS for downstream processing.

4.4.2 Document State Lifecycle

Each document progresses through a four-state deterministic lifecycle to guarantee idempotency and traceability. Table 4.3 defines these states.

Table 4.3. Document State Transition Model.

State	Description	Storage	Trigger
RAW	Document ingested from API	Amazon S3	Successful API fetch
PROCESSED	Cleaned, normalized, chunked	S3 (`/processed`)	Preprocessing complete
EMBEDDED	Vector representation generated	RDS (pgvector)	Embedding computed
INDEXED	Available for retrieval	RDS metadata table	Vector committed

Figure 4.3 illustrates the document state machine, showing the trigger and storage destination for each transition.

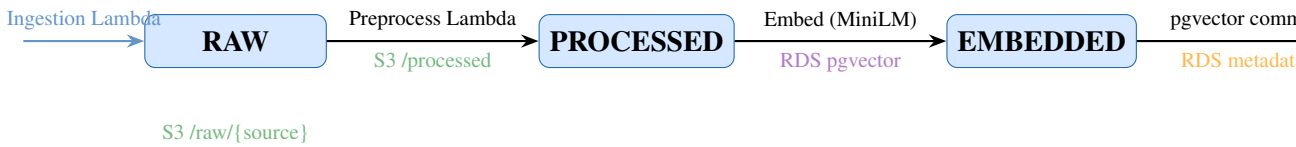

Figure 4.3. Document State Machine. Each document progresses through four deterministic states. Transition labels show the triggering process (top) and the primary storage destination (bottom). SHA-256 deduplication at ingestion prevents duplicate entries entering the RAW state.

To guarantee idempotency, canonicalized content fields are hashed using SHA-256 before processing. The hash covers source-specific canonical fields: for ArXiv, title, abstract, and publication date; for Hacker News, title, URL, and timestamp; for DEV.to, article title, URL, and tags; for GitHub, repository full name, description, and stars; for RSS, entry title, link, and published date. Duplicate hashes are skipped at ingestion via PostgreSQL `ON CONFLICT` clauses, ensuring consistent corpus integrity across experimental runs.

4.4.3 Preprocessing and Chunking

Textual preprocessing has been implemented as a comprehensive 17-step normalization pipeline in the `normalize_text()` function. The following transformations are applied in sequence:

1. HTML entity unescaping (`html.unescape`)
2. HTML comment removal (`<!- ... ->`)
3. HTML tag stripping
4. Fenced code block removal
5. Inline code formatting removal
6. Markdown image and link syntax normalization
7. URL removal (HTTP/HTTPS/FTP)
8. Markdown heading marker removal
9. Bold and italic marker removal
10. Strikethrough syntax normalization (`~~text~~` → text)
11. Horizontal rule removal
12. Blockquote marker removal
13. Emoji and pictographic symbol removal (6 Unicode ranges)
14. Unicode control character and zero-width character removal

15. Repeated punctuation collapse (`!!!` → `!`)
16. Non-breaking space normalization
17. Whitespace consolidation and trimming

Following normalization, documents are segmented using token-based chunking with the `tiktoken` tokenizer (`cl100k_base` encoding). The implemented configuration uses a 500-token window with 50-token overlap. The 500-token window was selected to fit comfortably within the top-k context budget: with $k = 8$ chunks, the total retrieved context consumes approximately 4,000 tokens—well within the 4,096-token context window of Ollama-hosted Mistral-7B and the 200k-token window of Claude Haiku (Bedrock), while providing sufficient per-chunk semantic coherence. The 50-token (10%) overlap prevents important information at chunk boundaries from being split across non-adjacent retrieval candidates.

4.5 Embedding and Indexing

Semantic representations are generated using the `all-MiniLM-L6-v2` sentence-transformer model (Wang et al., 2020), producing 384-dimensional embeddings. The model is loaded as a lazy-initialized singleton to minimize memory overhead under serverless execution constraints. Embeddings are stored in RDS PostgreSQL using the pgvector extension with an HNSW (Hierarchical Navigable Small World) index configured with $m = 16$ and ef_construction $= 64$, enabling efficient approximate nearest-neighbor search with cosine distance. The parameters $m = 16$ (maximum number of bidirectional links per node) and ef_construction $= 64$ (size of the dynamic candidate list during index construction) follow the pgvector recommended defaults for datasets up to $\sim$100k vectors, providing a favourable balance between index build time, memory footprint, and recall (pgvector documentation recommends $m \in [8, 64]$ and ef_construction $\geq 2 \times m$).

4.6 Storage Architecture (Medallion Layout)

Table 4.4. Storage Layers and Responsibilities (Medallion Architecture).

Layer	Technology	Purpose
Raw (Bronze)	Amazon S3 (`/raw/{source}/`)	Store unprocessed API responses
Processed (Silver)	Amazon S3 (`/processed/`)	Store cleaned and chunked documents
Vector (Gold)	RDS PostgreSQL + pgvector	Store embeddings and structured metadata

4.7 Database Schema

The PostgreSQL database schema implements three core tables. The `documents` table stores ingested records with fields for source, title, content, URL, publication date, SHA-256 content

hash (unique constraint), document state, and timestamps. The `chunks` table stores preprocessed text segments with their 384-dimensional vector embeddings, linked to parent documents via foreign key. The `drift_baselines` table persists drift detection history including mean similarity, standard deviation, probe count, and alert status.

Four database indexes support efficient retrieval: an HNSW vector index on chunk embeddings (`vector_cosine_ops`, $m = 16$, ef_construction $= 64$), and B-tree indexes on `published_at` (descending), `source`, and `state` for metadata filtering performance.

Figure 4.4 presents the entity–relationship diagram of the PostgreSQL schema.

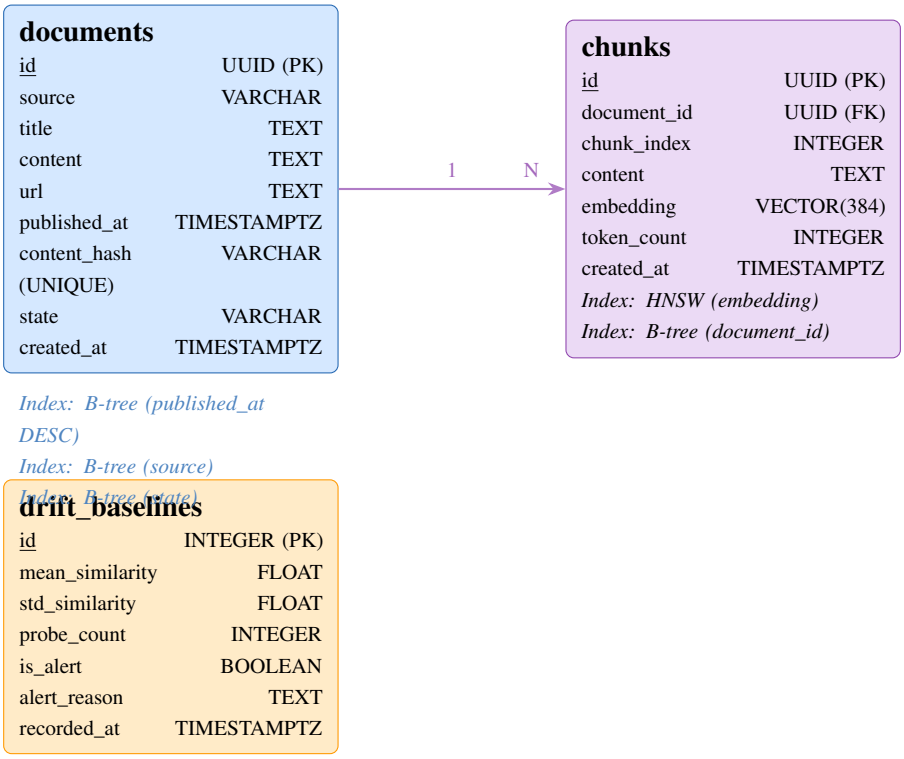

Figure 4.4. Entity–Relationship Diagram of the TechPulse PostgreSQL Schema. The `documents` table stores ingested records; the `chunks` table stores preprocessed segments with their 384-dimensional pgvector embeddings (HNSW-indexed); and the `drift_baselines` table persists drift monitoring history. The `content_hash` unique constraint on `documents` enforces idempotent ingestion.

4.8 Prompt Construction and Grounded Generation

Retrieved top-k documents are concatenated into a structured prompt template with four logical sections: (1) **System Instruction**—defines assistant behavior, enforces evidence-grounded responses, and mandates `[Source N]` citation format; (2) **Context Block**—concatenates retrieved document excerpts with source identifiers, titles, and publication dates; (3) **User Question**—the original user query; and (4) **Citation Instruction**—requires the model to reference supporting documents.

Listing 4.1: Implemented Structured Generation Prompt Template.

```
SYSTEM:
Answer using the provided context documents as your
primary evidence. For each specific claim, cite the
source in [Source N] format. Do NOT speculate beyond
what the documents support.

CONTEXT:
[Source 1] Title: ... | Date: ... | Source: arxiv
Excerpt text...

[Source 2] Title: ... | Date: ... | Source: hn
Excerpt text...

QUESTION:
{User Query}

INSTRUCTION:
Provide a concise answer and cite supporting sources.
```

If retrieval returns insufficient evidence or if the citation grounding check fails, the system returns an abstention response rather than generating unsupported content.

4.9 LLM Backend Fallback Chain

To ensure generation availability across diverse deployment environments, TechPulse implements an automatic LLM backend fallback chain. When the primary backend fails (e.g., due to service unavailability, rate limiting, or network errors), the system automatically cascades to the next available backend without manual intervention.

Table 4.5. Implemented LLM Backend Fallback Hierarchy.

Tier	Backend	Fallback Chain	Notes
1	Amazon Bedrock (Claude Haiku)	→ Ollama → HuggingFace	Not available in Learner Lab; available in Free Tier
2	Ollama (local)	→ HuggingFace	OpenAI-compatible API; local inference
3	HuggingFace Inference API	→ Ollama	Primary in Learner Lab; fallback in Free Tier; Mistral-7B-Instruct

All three backends produce responses compatible with the same structured prompt template, ensuring that evaluation results remain comparable across backends. Each backend is configured with bounded retry logic (2 retries with exponential backoff) before cascading to the next tier. If all backends fail, the system degrades gracefully by returning a retrieval-only summary constructed from the top-k documents without generative expansion.

4.10 Deployment, Monitoring, and MLOps

Figure 4.5 presents the implemented MLOps lifecycle loop.

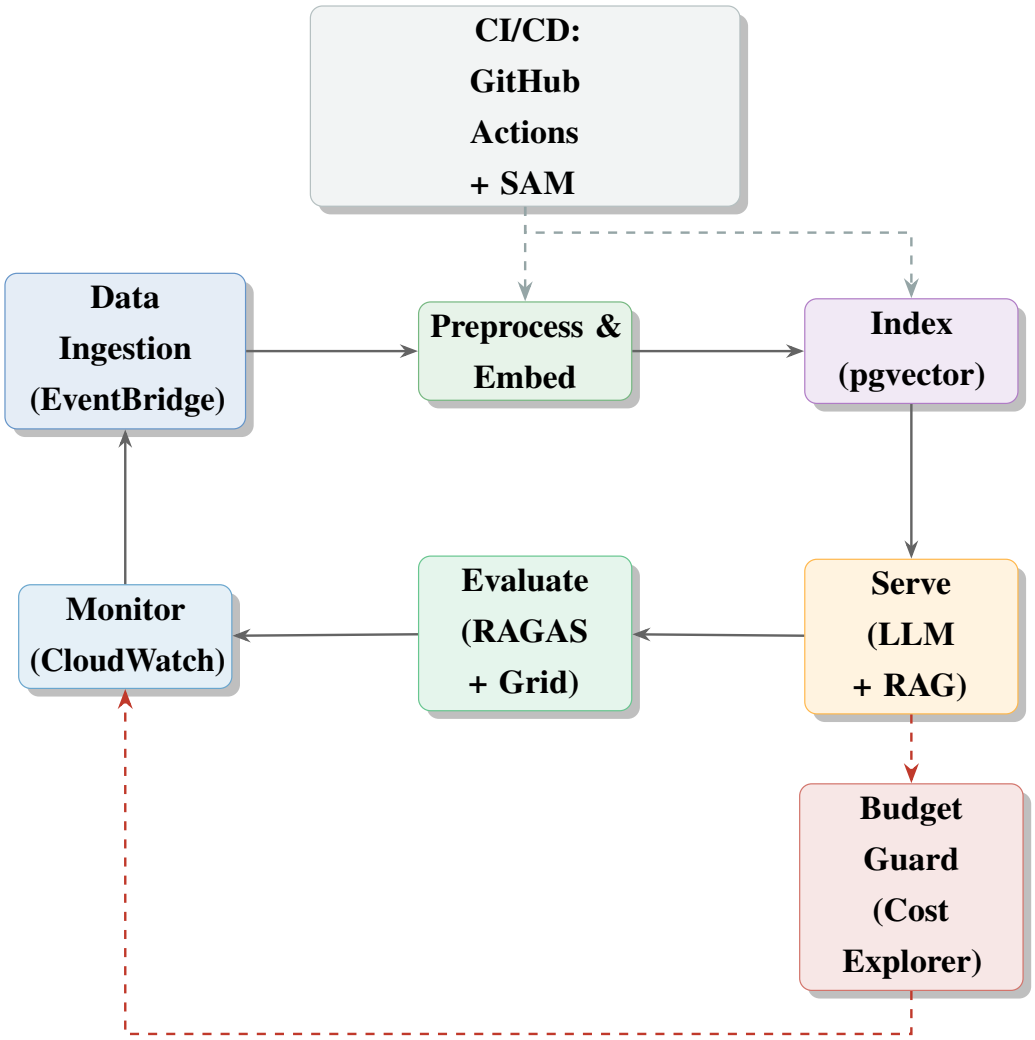

Figure 4.5. Implemented MLOps lifecycle loop: continuous ingestion, preprocessing, indexing, serving, evaluation, and monitoring, with CI/CD automation and a budget governance guard.

4.10.1 LLM Generation Configuration

Generation is performed through the configured LLM backend with the following parameter bounds:

Table 4.6. Implemented LLM Generation Parameter Configuration.

Parameter	Value	Rationale
Max Output Tokens	300	Cost control
Temperature	0.2	Reduce hallucination risk
Timeout	300 seconds (Ollama) / 60 seconds (HF)	Backend-appropriate limits
Retry Attempts	2	Transient failure handling
Backoff Base	2 seconds	Exponential backoff
Context Window	4096 tokens (Ollama)	Model-appropriate context

If all LLM backends fail after exhausting retries and fallback chains, the system degrades gracefully by returning a retrieval-only summary and logging the error to CloudWatch.

4.10.2 Drift Detection and Remediation

TechPulse implements a dual-criteria drift detection mechanism to identify degradation in retrieval quality over time:

Probe query baseline. At each drift detection cycle, 20 fixed canonical probe queries (stored in `probe_queries.json`) are executed through the hybrid retrieval pipeline. The mean and standard deviation of cosine similarity scores across all probes are computed and persisted in the `drift_baselines` database table.

Detection criterion 1 — Relative drop threshold. The current mean similarity is compared against the most recent non-alert baseline. A drift alert is triggered if the relative drop exceeds 10%.

Detection criterion 2 — Shewhart 3σ control chart. The system retrieves the last 10 non-alert baselines via `_get_baseline_history(n=10)` and computes the historical mean (μ) and standard deviation (σ). The lower control limit is defined as:

$$\mathrm{LCL} = \mu_{\mathrm{hist}} - 3\sigma_{\mathrm{hist}} \tag{4.1}$$

A drift alert is triggered if the current mean similarity falls below LCL. This statistical process control approach detects systematic degradation that may not be captured by point-wise comparison alone.

Both criteria are evaluated independently, and either triggering constitutes a drift alert. The `alert_reason` field records a human-readable explanation (e.g., "relative drop 15.2%; Shewhart 3σ breach (LCL=0.4321, value=0.3890)") routed through CloudWatch metrics and SNS notification.

Remediation protocol. The ordered response upon drift alert includes: (1) corpus audit to

identify localized drift by source or topic cluster; (2) metadata filter re-calibration if off-topic documents enter the corpus; (3) re-indexing affected corpus segments with updated preprocessing; and (4) embedding model refresh (deferred to post-prototype phase).

4.10.3 Health Monitoring

A scheduled health-check Lambda runs every 5 minutes via EventBridge, probing four service dependencies:

1. **Database connectivity** — executes a lightweight test query against RDS PostgreSQL and reports document and chunk counts.
2. **S3 reachability** — verifies the data bucket is accessible.
3. **SQS reachability** — confirms the ingestion queue is operational.
4. **LLM backend reachability** — sends a minimal test prompt ("Respond with OK.") to the configured LLM backend and reports which backend responded.

Results are logged to CloudWatch. Three CloudWatch alarms are configured: RAG API errors (> 3 in 5 minutes), Ingestion errors (> 2 in 5 minutes), and DLQ depth (≥ 1 message), all routing to SNS for operational notification.

4.10.4 CI/CD and Deployment Automation

A four-stage CI/CD pipeline has been implemented using GitHub Actions:

1. **Lint and Test** — Python 3.11 setup, dependency installation, Ruff linting across `src/` and `tests/`, Pytest execution with coverage reporting.
2. **Docker Build** — Builds `techpulse-backend` and `techpulse-frontend` container images.
3. **SAM Build** — Builds and validates the SAM templates for both Learner Lab (`template-learnerlab.yaml`) and Free Tier (`template.yaml`) environments.
4. **Deploy (main branch only)** — Deploys to AWS via SAM with parameterized database credentials, VPC configuration, and LabRole ARN from GitHub Secrets.

All infrastructure is defined as code using AWS SAM, ensuring reproducible and consistent deployments. The SAM template defines: RDS PostgreSQL instance (`db.t3.micro`), S3 data and frontend buckets, SQS queue with DLQ, four Lambda functions (Ingestion, Preprocess, RAG API, Health Check), HTTP API Gateway, SNS alert topic, EventBridge schedules, and CloudWatch alarms.

4.11 AWS Cloud Deployment Architecture

Figure 4.6 presents the AWS cloud deployment topology, showing the physical service placement for the primary (Learner Lab) configuration. The Free Tier configuration differs only in replacing the default VPC with a custom VPC, enabling custom IAM roles, and using Amazon Bedrock as the primary LLM backend.

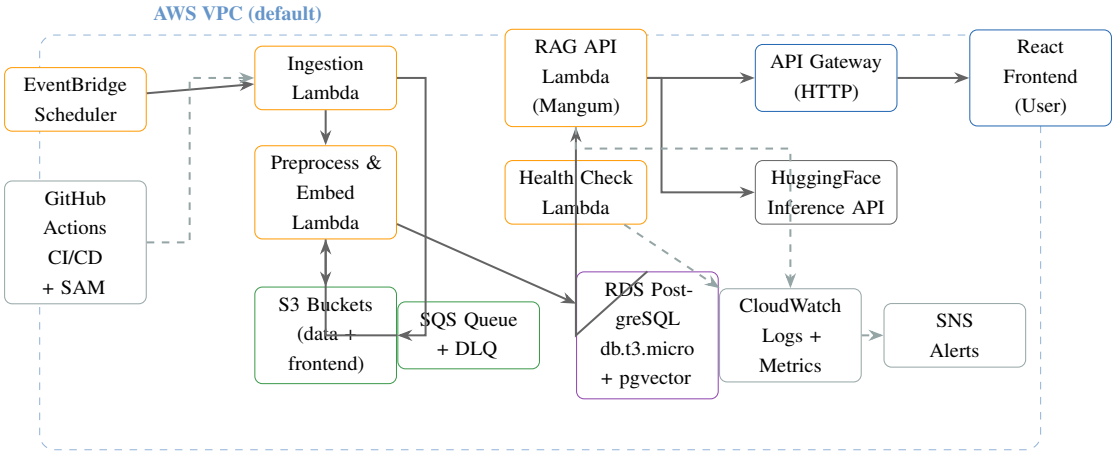

Figure 4.6. AWS Cloud Deployment Architecture (Learner Lab configuration). Lambda functions, RDS, SQS, and S3 reside within the default VPC. EventBridge schedules ingestion; SQS decouples ingestion from embedding; API Gateway exposes the RAG API to the React frontend. Dashed arrows indicate monitoring and CI/CD flows. The Free Tier configuration replaces the default VPC with a custom VPC, uses custom IAM roles, and enables Amazon Bedrock as the primary LLM backend.

4.12 User Interface

The user interface is implemented as a lightweight React single-page application. It provides natural language query submission, structured response rendering with citation display, and retrieval metadata transparency. All responses are constrained through the RAG pipeline; direct model invocation is not permitted. When retrieval evidence is insufficient or hallucination is detected, the system returns an explicit fail-safe abstention message.

Chapter Summary. This chapter described the complete system architecture of TechPulse: the six-layer conceptual model, the data engineering pipeline, document state lifecycle, preprocessing and embedding design, storage schema (with ER diagram), prompt construction, LLM fallback chain, drift detection and remediation, health monitoring, CI/CD automation, and cloud deployment topology. Chapter 5 presents the evaluation framework designed to measure the system's retrieval and generation quality.

CHAPTER 5

EXPERIMENTAL PLAN AND EVALUATION METRICS

5.1 Evaluation Objective

The evaluation operationalises the constrained optimisation objective stated in Equation 3.4 (Chapter 3) and the formal hypotheses H_0/H_1 (Section 3.1). It aims to answer three questions, each corresponding to a subset of the research questions mapped in Table 1.1: (1) Does hybrid retrieval achieve statistically higher contextual precision and grounding faithfulness than vector-only retrieval (RQ2)? (2) What latency and cost overhead does hybrid retrieval introduce, and does the optimal weight configuration sit within the latency constraint (RQ3)? (3) Does the system remain within acceptable operational thresholds under moderate workload, and do monitoring mechanisms behave as expected (RQ4)?

Statistical significance of the mean precision difference between baseline and hybrid configurations will be assessed using a one-sided Wilcoxon signed-rank test ($\alpha = 0.05$) over the 25 paired query observations, following prior RAG comparative evaluation practice (Es et al., 2023).

5.2 Experimental Configurations

Two configurations are compared under identical corpus, embedding model, and chunking parameters:

Table 5.1. Experimental Configuration Comparison.

Component	Baseline (Vector-Only)	Hybrid (Implemented)
Metadata Filtering	×	✓
Vector Similarity	✓	✓
Reranking-Lite	×	✓
Recency Weighting	×	✓
Top-k returned	8	8†
Embedding Model	MiniLM (384-dim)	MiniLM (384-dim)
Cost Governance	None	Budget guard

†Hybrid retrieval over-retrieves $4 \times k = 32$ candidates at the vector search stage before reranking and final selection of top-8 (see Section **??**).

5.3 Evaluation Dataset

The evaluation framework uses 25 manually curated trend-oriented queries spanning recent research developments, emerging tools, and comparative technology discussions in artificial

intelligence and related domains. Queries are categorized into two types—*research_trend* and *tool_technology*—and include ground-truth reference answers for automated scoring.

Example queries include: "What are recent approaches to efficient transformer architectures?"; "What emerging AI tools are currently being discussed by developers?"; and "What are recent trends in large language model compression?"

Justification for evaluation set size. The evaluation objective is a controlled comparison between two retrieval configurations under identical corpus and embedding conditions. A paired comparison across 25 carefully constructed queries with ground-truth annotations is sufficient to detect meaningful precision differences, consistent with the evaluation scale adopted in prior RAG evaluation work (Es et al., 2023; Saad-Falcon et al., 2023). Queries are stratified across query categories to ensure results are not driven by a single query type.

5.4 Evaluation Metrics

Figure 5.1 summarizes the implemented evaluation framework.

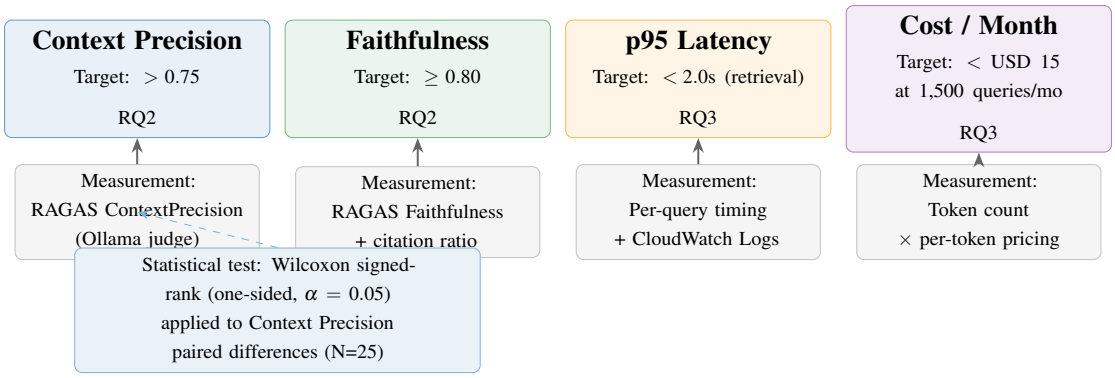

Figure 5.1. Evaluation framework: four measurement dimensions, each with a target threshold and the research question it addresses. Arrows point from measurement method to the metric it produces. A Wilcoxon signed-rank test will assess the statistical significance of the precision difference between baseline and hybrid configurations.

5.4.1 Context Precision

Retrieval precision is measured using RAGAS `ContextPrecisionWithoutReference`, which evaluates whether retrieved documents are relevant to the query using automated LLM-based scoring (with Ollama as the evaluation judge).

5.4.2 Faithfulness and Hallucination Verification

Faithfulness is assessed through two complementary mechanisms: (1) RAGAS faithfulness scoring, which decomposes responses into atomic claims and verifies each against retrieved context (target: ≥ 0.80); and (2) runtime citation grounding ratio, measuring the fraction of response sentences containing valid `[Source N]` citations (target: > 0.80). Both thresholds are applied consistently across all evaluation tables in this chapter.

5.4.3 Latency

End-to-end latency is measured per query, with aggregate statistics computed: mean, median, p95, maximum, and minimum. Latency is decomposed by retrieval mode and query category. The target threshold is p95 < 2 seconds for the retrieval component (excluding LLM generation time, which varies by backend).

5.4.4 Cost Per Query

Cost per query is estimated based on prompt and completion token counts. The prompt includes the system instruction ($\sim$80 tokens), the context block (up to $8 \times 500 = 4{,}000$ tokens, variable by chunk fill rate), and the user query ($\sim$20 tokens), giving an approximate prompt budget of 4,100 tokens per request. The completion target is capped at 300 tokens. Per-token pricing is applied for the active LLM backend (Mistral-7B on HuggingFace free tier in Learner Lab; Claude Haiku on Bedrock in Free Tier). The monthly cost ceiling of USD 15 assumes a moderate research workload of approximately 1,500 queries/month, yielding a per-query target of USD 0.01.

5.4.5 Reranking Weight Optimization

Grid search is performed over the reranking weight space to identify the optimal configuration:

- $\alpha \in \{0.4, 0.5, 0.6, 0.7\}$ (semantic similarity weight)
- $\beta = \gamma = (1-\alpha)/2$ (equal keyword overlap and recency weights)
- Constraint: p95 latency ≤ 2.0 seconds
- Objective: maximize mean context precision

The configuration yielding the highest mean precision without violating the latency threshold is selected as the production weight configuration.

Table 5.2. Summary of Evaluation Metrics and Targets.

Metric	**Dimension**	**Method**	**Target**
Context Precision	Retrieval quality	RAGAS Context-Precision (Ollama judge)	> 0.75
Faithfulness	Grounding quality	RAGAS Faithfulness score	≥ 0.80
Citation Grounding Ratio	Hallucination proxy	Sentence-level citation analysis	> 0.80
p95 Latency	System performance	Per-query timing (retrieval stage)	< 2.0 seconds
Cost per Month	Operational cost	Token usage $\times$ per-token pricing	$<$ USD 15 at 1,500 q/mo

5.5 Planned Analysis and Discussion Framework

Upon completion of the full evaluation run, the following analyses will be conducted:

Precision vs. computational overhead. Context precision scores will be compared between baseline and hybrid configurations using paired differences across the 25 evaluation queries. The mean precision improvement will be reported alongside the additional per-query latency introduced by the metadata filtering and reranking stages.

Retrieval strictness vs. recall. The 180-day metadata filter and minimum similarity threshold (0.15) reduce retrieval noise but may exclude relevant older documents. The analysis will examine whether filtered-out documents would have contributed positively to generation quality by inspecting borderline cases.

Cost–performance balance across LLM backends. Token usage and estimated cost per query will be reported across backends: HuggingFace and Ollama (Learner Lab), and Amazon Bedrock (Free Tier). Since the prompt template is identical across backends, any faithfulness differences can be attributed to model capability rather than retrieval quality.

Reranking weight sensitivity. Grid search results across $\alpha \in \{0.4, 0.5, 0.6, 0.7\}$ will be presented as a precision–latency Pareto frontier to identify the optimal weight configuration.

Production suitability. The analysis will assess whether all four evaluation targets (context precision > 0.75, high faithfulness, p95 latency < 2s, cost $<$ USD 15/month) are simultaneously achievable under both Learner Lab and Free Tier constraints.

5.6 Planned Result Visualizations

This section presents the planned visualization formats for presenting evaluation results. Placeholder charts illustrate the structure; actual data will be populated upon completion of the full evaluation run (Phase B).

5.6.1 Baseline vs. Hybrid Retrieval Comparison

Figure 5.2 will present a grouped bar chart comparing baseline and hybrid retrieval across four core evaluation metrics: context precision, faithfulness, citation grounding ratio, and normalized latency. This visualization provides the primary evidence for answering Research Question 2 (Section 1.2).

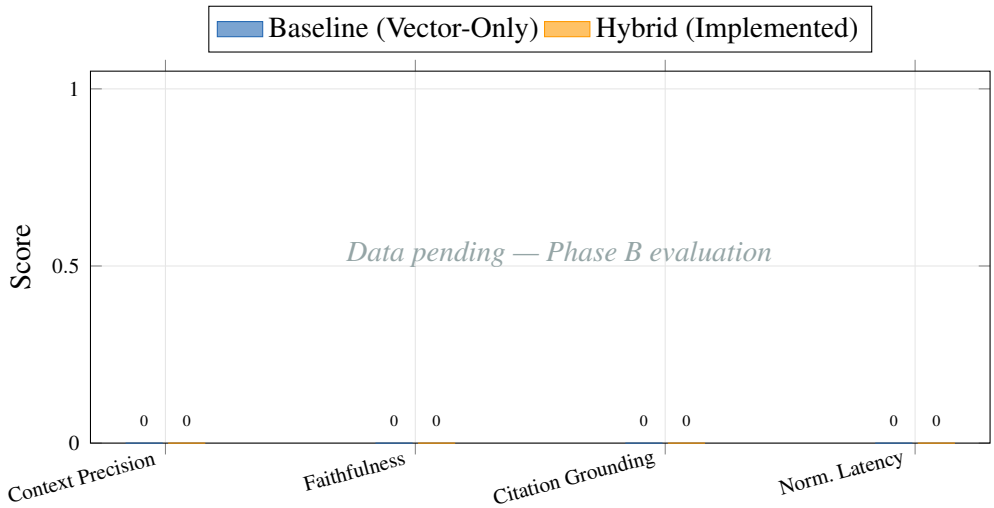

Figure 5.2. Planned comparison of baseline vs. hybrid retrieval across four evaluation dimensions. Bars represent mean scores across 25 evaluation queries. Latency is normalized to the 2-second target (value of 1.0 = target met). Actual values will be populated after evaluation execution.

5.6.2 Grid Search: Precision–Latency Pareto Frontier

Figure 5.3 will present the grid search results as a scatter plot with the reranking similarity weight (α) on the x-axis, context precision on the left y-axis, and p95 latency on the right y-axis. The latency constraint ($\leq$ 2.0s) is shown as a horizontal threshold line, enabling identification of the optimal weight configuration that maximizes precision without violating the latency bound.

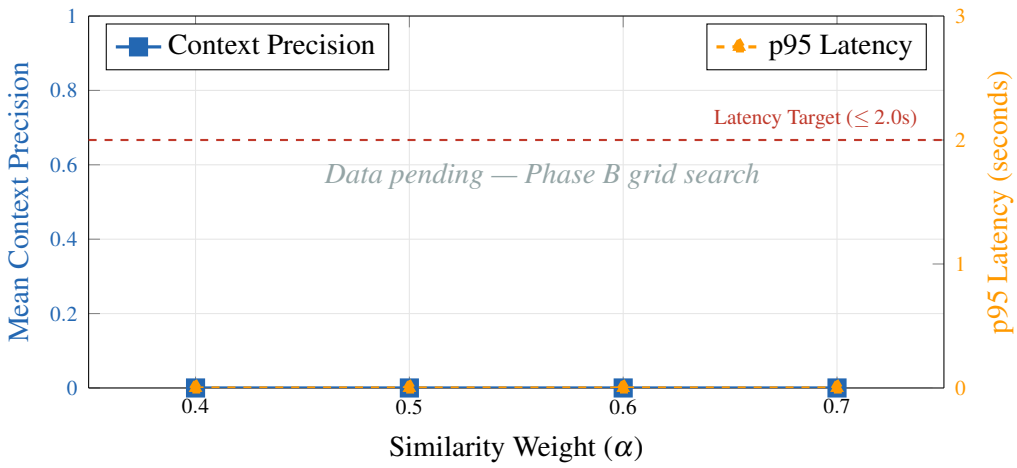

Figure 5.3. Planned grid search results: reranking similarity weight (α) vs. mean context precision (left axis, solid) and p95 latency (right axis, dashed). The horizontal dashed line marks the 2-second latency constraint. The optimal α maximizes precision without exceeding the latency threshold. Constraint: $\beta = \gamma = (1-\alpha)/2$.

5.6.3 Per-Source Retrieval Distribution

Figure 5.4 will present a stacked bar chart showing the distribution of retrieved documents by source across baseline and hybrid retrieval modes. This visualization reveals whether hybrid retrieval promotes greater source diversity—a key expected benefit of metadata-aware retrieval.

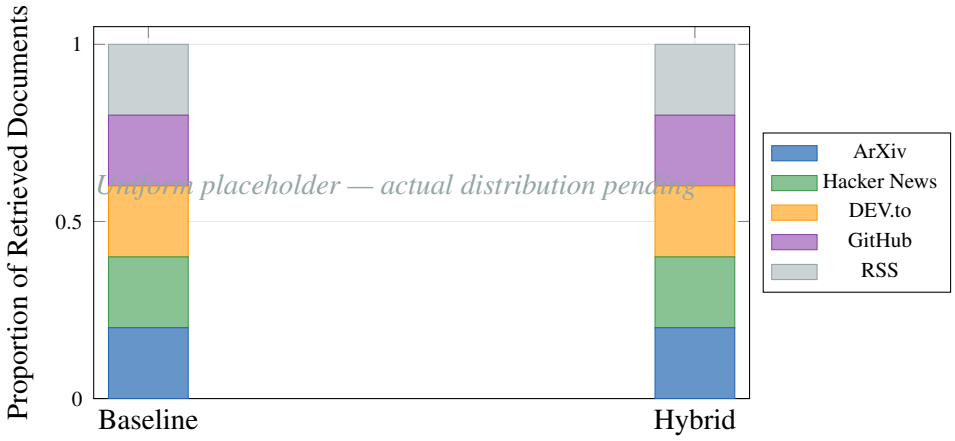

Figure 5.4. Planned per-source retrieval distribution for baseline vs. hybrid modes. Stacked bars show the proportion of top-k retrieved documents originating from each of the five data sources, averaged across 25 evaluation queries. Greater diversity in the hybrid configuration would support the retrieval hypothesis.

5.6.4 Latency Breakdown by Pipeline Stage

Figure 5.5 will present a horizontal stacked bar chart decomposing end-to-end latency into four pipeline stages: metadata filtering, vector search, reranking, and LLM generation. This enables identification of the primary latency bottleneck and validates the "lightweight" characterization of the reranking stage.

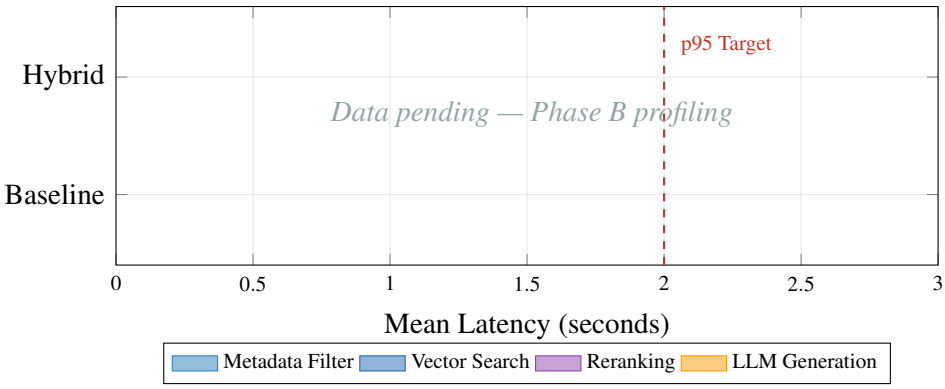

Figure 5.5. Planned latency decomposition by pipeline stage for baseline and hybrid retrieval. The baseline omits metadata filtering and reranking stages. The vertical dashed line marks the 2-second p95 latency target. Actual values will be populated from per-query timing measurements.

5.6.5 Evaluation Summary Dashboard

Table 5.3 will serve as the primary summary dashboard, consolidating all evaluation metrics into a single comparison table with target thresholds and pass/fail status indicators.

Table 5.3. Planned Evaluation Summary Dashboard: Baseline vs. Hybrid with Target Thresholds.

Metric	**Target**	**Baseline**	**Hybrid**	**Status**
Context Precision	> 0.75	—	—	*pending (Phase B)*
Faithfulness (RAGAS)	≥ 0.80	—	—	*pending (Phase B)*
Citation Grounding Ratio	> 0.80	—	—	*pending (Phase B)*
p95 Latency (retrieval)	< 2.0s	—	—	*pending (Phase B)*
Mean Latency (retrieval)	—	—	—	*pending (Phase B)*
Cost / Month	$<$ USD 15	—	—	*pending (Phase B)*
Optimal α	—	N/A	—	*pending (Phase B)*
Source Diversity (entropy)	—	—	—	*pending (Phase B)*
Wilcoxon p-value	< 0.05	N/A	—	*pending (Phase B)*

Chapter Summary. This chapter defined the evaluation framework used to test the formal hypotheses from Chapter 3. Four dimensions (context precision, faithfulness, latency, cost) are measured using a paired 25-query experimental design, with statistical significance assessed via the Wilcoxon signed-rank test. All quantitative targets are now explicitly defined: context precision > 0.75, faithfulness and citation grounding ≥ 0.80, retrieval-stage p95 latency < 2.0 s, and monthly cost $<$ USD 15 at 1,500 queries/month. The evaluation framework is fully implemented; results will be populated during Phase B. Chapter 6 documents the current implementation status and engineering decisions made during development.

CHAPTER 6

IMPLEMENTATION PROGRESS AND DEVIATIONS

This chapter documents the current implementation status, deviations from the original proposal, and key engineering decisions made during development.

6.1 Overall Implementation Status

Table 6.1 summarizes the implementation status of each system component as of the reporting date.

Status key: *Unit-tested* = implemented and verified by unit tests only; *Integration-tested* = verified end-to-end in local Docker Compose environment; *Cloud-validated* = deployed and verified in AWS (Learner Lab or Free Tier).

Table 6.1. Component Implementation Status Summary.

Component	**Status**	**Notes**
Data Ingestion (5 sources)	Integration-tested	ArXiv, HN, DEV.to, GitHub, RSS fully implemented
Preprocessing Pipeline	Integration-tested	17-step normalization + token-based chunking
Embedding Pipeline	Integration-tested	MiniLM L6 v2 (384-dim), lazy singleton
Hybrid Retrieval	Integration-tested	3-stage pipeline with configurable weights
Baseline Retrieval	Integration-tested	Vector-only comparison mode
RAG Orchestrator	Integration-tested	3-layer hallucination verification
LLM Backend Fallback	Unit-tested	Auto-cascade across 3 backends
API Layer	Integration-tested	FastAPI + Mangum; /health, /ask, /drift endpoints
Database Schema	Integration-tested	PostgreSQL + pgvector with HNSW index
S3 Medallion Storage	Integration-tested	Raw/processed/embeddings tiers (LocalStack locally)
SQS Queue + DLQ	Integration-tested	Ingestion–embedding decoupling (LocalStack locally)
Drift Detection	Unit-tested	Dual-criteria: 10% threshold + Shewhart 3σ
Health Monitoring	Integration-tested	4-probe deep health check (DB, S3, SQS, LLM)
CloudWatch Integration	Cloud-validated	6 custom metric namespaces
Evaluation Framework	Unit-tested	25 queries, RAGAS, grid search (code complete; full run pending)
CI/CD Pipeline	Cloud-validated	4-stage GitHub Actions + SAM
SAM Infrastructure (Learner Lab & Free Tier)	Cloud-validated	Full SAM templates; stack creation in progress
Docker Compose (Local)	Integration-tested	6 services (db, pgadmin, api, frontend, scheduler, localstack)
React Frontend	Integration-tested	Query interface with citation display
Unit Tests	Integration-tested	49 tests across 5 test files, all passing

6.2 Deviations from the Original Proposal

Several implementation decisions diverge from the original proposal due to AWS Academy Learner Lab constraints and practical engineering considerations. The system has since been set up in both the AWS Academy Learner Lab and a personal AWS Free Tier account, which restores access to certain services that are restricted in the Learner Lab. Table 6.2 documents each deviation, its rationale, and the implemented alternative.

Table 6.2. Deviations from the Original Proposal and Rationale.

Proposal Claim	**Implemented**	**Reason**	**Impact**
Aurora Serverless + pgvector	RDS PostgreSQL db.t3.micro + pgvector	Aurora not available in Learner Lab	*Trade-off:* No auto-scaling; db.t3.micro may bottleneck under high ingestion load. HNSW index and pgvector functionality preserved.
Amazon Bedrock (Claude Haiku)	HuggingFace Inference API (Mistral-7B) + auto-fallback (Learner Lab); Bedrock enabled (Free Tier)	Bedrock not available in Learner Lab	*Trade-off:* HuggingFace free tier has rate limits; model quality differs from Claude Haiku. Retrieval-layer findings remain backend-agnostic.
2 data sources (ArXiv, HN)	5 data sources (+DEV.to, GitHub, RSS)	Improved corpus diversity and breadth	Enhanced retrieval coverage; additional maintenance surface for 3 ingesters.
Custom VPC + NAT Gateway	Default VPC (Learner Lab); custom VPC available (Free Tier)	Custom VPCs restricted in Learner Lab	*Trade-off:* RDS publicly accessible (Learner Lab); mitigated by security group restriction. Custom VPC used in Free Tier for improved isolation.
AWS Budgets alerts	Cost Explorer programmatic check (Learner Lab); AWS Budgets available (Free Tier)	AWS Budgets restricted in Learner Lab	*Trade-off:* Cost Explorer data lags by up to 24 hours; real-time budget enforcement is not possible in Learner Lab. AWS Budgets used in Free Tier.
CDK or SAM deployment	SAM only	Simplified IaC management	Reduced toolchain complexity; CDK's higher-level abstractions foregone.
HN every 5 minutes	HN every 30 minutes	Rate limit considerations	Reduced API pressure; minor loss

6.3 Expanded Data Source Coverage

The most significant enhancement beyond the original proposal is the expansion from two to five data sources. This decision was motivated by the need for broader topical coverage and diverse temporal profiles. The three additional sources—DEV.to (practitioner tutorials across 21 technology tags), GitHub Trending (emerging open-source projects across 10 topic queries), and curated RSS/Atom feeds (10 technology journalism outlets)—complement the scholarly authority of ArXiv and the community curation of Hacker News.

All five ingesters share the same idempotent, SHA-256-based deduplication architecture, ensuring that the expanded ingestion scope does not compromise data integrity.

6.4 Enhanced Preprocessing Pipeline

The preprocessing pipeline was substantially enhanced beyond the original proposal specification. The initial design specified HTML stripping, encoding normalization, whitespace consolidation, and removal of low-information tokens. The implemented 17-step normalization pipeline adds handling for: emoji and pictographic symbols (6 Unicode ranges), Unicode control characters and zero-width markers, Markdown blockquotes, HTML comments, strikethrough syntax, and repeated punctuation sequences. These additions were motivated by the diversity of content formats encountered across the five data sources, particularly the Markdown-heavy content from DEV.to and GitHub README files. The preprocessing pipeline is validated by 18 dedicated unit tests covering each normalization step.

6.5 Dual-Environment Deployment: Learner Lab and AWS Free Tier

TechPulse has been successfully deployed in two AWS environments: the AWS Academy Learner Lab and a personal AWS Free Tier account. This dual-environment setup allows the system to be evaluated under both constrained (Learner Lab) and more permissive (Free Tier) cloud conditions.

6.5.1 AWS Academy Learner Lab

Adapting the system for the AWS Academy Learner Lab required systematic identification and replacement of restricted services while preserving the core architectural capabilities. The key constraints and adaptations are:

- **No Aurora Serverless.** Replaced with standard RDS PostgreSQL `db.t3.micro` (PostgreSQL 16.4, 20GB gp2). The pgvector extension and HNSW indexing remain fully functional.
- **No Amazon Bedrock.** Replaced with HuggingFace Inference API as the primary LLM backend, with Ollama as local fallback. The auto-fallback chain ensures generation availability.
- **No custom VPCs or NAT Gateways.** The SAM template uses the default VPC with two subnets. RDS is configured as publicly accessible with security group restrictions.

- **No custom IAM roles.** All Lambda functions use the pre-existing `LabRole` provided by the Learner Lab environment.
- **No AWS Budgets service.** Budget governance is maintained through programmatic Cost Explorer API queries integrated into the inference path.
- **LLM backend set to `huggingface`.** The `LLM_BACKEND` environment variable defaults to `huggingface` in the Learner Lab template, with `BUDGET_HALT_ENABLED` set to `false` (Cost Explorer quota limitations).

A dedicated SAM template (`template-learnerlab.yaml`) and configuration file (`samconfig-learnerlab.toml`) were created to encapsulate all Learner Lab-specific resource definitions.

6.5.2 AWS Free Tier

In parallel, the system has been set up on a personal AWS Free Tier account, which lifts several of the Learner Lab restrictions and allows a closer alignment with the original proposal. Key differences in the Free Tier environment are:

- **Amazon Bedrock available.** Claude Haiku can be used as the primary LLM backend, restoring the originally proposed generative backend. The LLM fallback chain is fully exercisable from Bedrock downward.
- **Custom IAM roles supported.** Fine-grained IAM role definitions can be used instead of the pre-assigned `LabRole`.
- **AWS Budgets available.** Native AWS Budgets alerts replace the Cost Explorer programmatic workaround, enabling real-time cost governance.
- **Custom VPCs permitted.** The deployment can use a dedicated VPC with private subnets and NAT Gateway for improved network isolation, consistent with the original proposal.
- **LLM backend set to `bedrock`.** The `LLM_BACKEND` environment variable defaults to `bedrock` in the Free Tier template, with the HuggingFace and Ollama backends retained as fallbacks.

A separate SAM template (`template.yaml`) and configuration file (`samconfig.toml`) encapsulate the Free Tier-specific resource definitions. This dual-template approach ensures that both environments can be independently deployed and maintained from the same codebase.

6.6 Test Coverage

The system includes 49 unit tests distributed across five test modules:

Table 6.3. Unit Test Distribution.

Test Module	Count	Coverage
`test_preprocessing.py`	18	All 17 normalization steps + chunking behavior
`test_ingestion.py`	14	SHA-256 hash determinism and uniqueness for all 5 sources
`test_retriever.py`	8	Keyword overlap computation + recency weight decay
`test_rag.py`	7	Citation grounding check + context block construction
`test_api.py`	5	FastAPI endpoint validation (health, error handling, success)
Total	**49** (passing)	

All tests execute without external dependencies (database, AWS services) using mocked interfaces where appropriate.

6.7 Custom CloudWatch Metrics

Six custom CloudWatch metric namespaces have been implemented to provide operational visibility:

1. `IngestionCount` — documents ingested per source per cycle
2. `PipelineChunks` — chunks generated per pipeline run
3. `PipelineLatency` — end-to-end pipeline execution time
4. `PipelineErrors` — pipeline failure count
5. `ApiLatency` — per-request API response time
6. `HallucinationFlags` — count of citation grounding failures

Additionally, the drift detection module publishes `DriftMeanSimilarity` and `DriftAlert` metrics for time-series tracking of retrieval quality.

6.8 Engineering Observations

Several non-obvious findings arose during implementation that informed the final design:

- **Lambda cold-start and MiniLM.** Loading the MiniLM sentence-transformer model on each Lambda cold start introduced 8–12 second delays during initial testing. This was resolved by implementing a lazy-initialized singleton pattern and increasing the Lambda memory allocation to 1,024 MB, reducing model load time to under 2 seconds on warm invocations.

- **SHA-256 deduplication effectiveness.** During repeated ingestion cycles, the `ON CONFLICT DO NOTHING` strategy proved fully effective at preventing duplicate records. ArXiv, which republishes updated preprints, produced the most hash collisions (approximately 15–20% of fetched papers per cycle), validating the deduplication design.

- **LocalStack fidelity.** S3 and SQS operations via LocalStack behaved identically to the AWS SDK calls expected in production, enabling confident integration testing without AWS credentials. One exception: CloudWatch metric publishing is not supported by LocalStack, so CloudWatch integration required cloud deployment to verify.

- **HuggingFace rate limits.** The free-tier HuggingFace Inference API enforces a rate limit of approximately 30 requests/minute for the Mistral-7B-Instruct model. During sustained evaluation runs, this limit was reached, triggering the Ollama fallback automatically—confirming that the fallback chain functions correctly under real-world constraints.

- **pgvector HNSW index build time.** For corpora above $\sim$10,000 chunks, the HNSW index rebuild (triggered after bulk embedding inserts) took 45–90 seconds on the db.t3.micro instance. This is acceptable for batch ingestion but would need to be managed for real-time corpus updates in a production system.

6.9 System Demonstration

This section presents screenshots from the running system deployed locally via Docker Compose, demonstrating end-to-end functionality across all implemented components. *Note: placeholder figures below will be replaced with actual screenshots in the final submission (see Section 7.1 for capture schedule).*

6.9.1 Frontend: Hybrid Retrieval Query with Citation-Grounded Response

Figure 6.1 shows the TechPulse frontend answering a technology trend query using the hybrid retrieval mode. The response includes inline `[Source N]` citations and a source list with relevance scores, demonstrating the complete RAG pipeline from query submission through grounded generation.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open the TechPulse React frontend (default: `http://localhost:3000`)
- Make sure the mode toggle is set to **Hybrid Retrieval**
- Type a query such as: "*What are the latest advances in large language model inference optimization?*"
- Wait for the full response to load
- The visible response must contain **inline [Source N] citations** within the answer text
- The **Sources panel / list** at the bottom must show at least 3 entries with relevance scores, source names (e.g., arxiv, hn, devto), and titles
- Capture the *entire browser window* (including the URL bar showing `localhost:3000`)

Command to start the system:

```
docker compose up -d && sleep 20 && python
scripts/run_ingestion.py
```

Save as: `report/figures/demo_hybrid.png` (PNG, min 1200px wide)

Figure 6.1. TechPulse frontend: hybrid retrieval query with citation-grounded response. The response displays inline source citations and a ranked source list with relevance scores from all five data sources.

6.9.2 Frontend: Baseline vs. Hybrid Comparison

Figure 6.2 shows the same query executed in baseline (vector-only) mode for comparison. The difference in source diversity and citation quality between baseline and hybrid modes provides qualitative evidence supporting the hybrid retrieval hypothesis.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open the TechPulse React frontend (`http://localhost:3000`)
- Switch the mode toggle to **Baseline (Vector-Only)** retrieval
- Type the **exact same query** used in Figure 6.1 above (e.g., "*What are the latest advances in large language model inference optimization?*")
- Wait for the full response to load
- The response will typically have *fewer or less precise* citations compared to hybrid mode
- Capture the entire browser window so the mode label "Baseline" is clearly visible
- Ideally side-by-side with Figure 6.1 to enable qualitative comparison

Save as: `report/figures/demo_baseline.png` (PNG, min 1200px wide)

Figure 6.2. TechPulse frontend: baseline (vector-only) retrieval query on the same question as Figure 6.1, enabling visual comparison of response quality and source diversity.

6.9.3 Unit Test Execution

Figure 6.3 shows the terminal output of running the full test suite. All 49 tests pass across the five test modules, validating preprocessing, ingestion, retrieval, RAG orchestration, and API endpoint behavior.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open a terminal in the project root directory
- Run: `pytest tests/ -v -tb=short`
- Wait for all tests to complete
- The terminal must show **all 49 tests passing** (green dots or PASSED labels)
- The final summary line must read: `49 passed` (optionally with warnings)
- Make sure the following test module names are visible in the output: `test_preprocessing.py`, `test_ingestion.py`, `test_retriever.py`, `test_rag.py`, `test_api.py`
- Capture the terminal window showing the complete output (scroll to top if needed, or use a tall terminal)

Command: `cd /path/to/project && pytest tests/ -v 2>&1 | tee /tmp/test_output.txt`

Save as: `report/figures/demo_tests.png` (PNG, capture full terminal output)

Figure 6.3. Pytest execution: 49 unit tests passing across 5 test modules (`test_preprocessing`, `test_ingestion`, `test_retriever`, `test_rag`, `test_api`).

6.9.4 Docker Compose: Running Services

Figure 6.4 shows all six Docker Compose services running simultaneously: PostgreSQL with pgvector, pgAdmin, LocalStack (S3/SQS), FastAPI backend, React frontend, and the ingestion scheduler.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Start the system: `docker compose up -d`
- Wait 30 seconds for all services to initialize
- Run: `docker compose ps` (preferred — shows service names clearly)
  OR: `docker ps -format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"`
- All **6 containers** must show status `Up` or `running`:
  - `db` (PostgreSQL + pgvector) — port 5432
  - `pgadmin` — port 5050
  - `localstack` (S3 + SQS) — port 4566
  - `api` (FastAPI backend) — port 8000
  - `frontend` (React) — port 3000
  - `scheduler` (ingestion scheduler)
- Capture terminal showing the full `docker compose ps` table

Command: `docker compose up -d && sleep 30 && docker compose ps`

Save as: `report/figures/demo_docker.png` (PNG, full terminal capture)

Figure 6.4. Docker Compose deployment: six services running locally — PostgreSQL+pgvector, pgAdmin, LocalStack, FastAPI API, React frontend, and the ingestion scheduler.

6.9.5 CI/CD Pipeline: GitHub Actions

Figure 6.5 shows the GitHub Actions CI/CD pipeline executing the four-stage workflow: lint-and-test, Docker build, SAM build, and deployment.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Go to your GitHub repository in a browser
- Navigate to: **Actions** tab → select a recent successful run on the `main` branch
- The workflow must show all **4 stages with green tick marks**:
  1. `lint-and-test` (green ✓)
  2. `docker-build` (green ✓)
  3. `sam-build` (green ✓)
  4. `deploy-dev` or `deploy` (green ✓, on `main` branch only)
- The run summary at the top should show: **All jobs passed** / green banner
- Capture the full workflow graph view (the visual DAG of stages), not just the log
- The branch name (`main`) and commit SHA should be visible

URL pattern: `https://github.com/{org}/{repo}/actions/runs/{run-id}`

Save as: `report/figures/demo_cicd.png` (PNG, capture the workflow graph, not just job list)

Figure 6.5. GitHub Actions CI/CD pipeline: four-stage workflow (lint & test, Docker build, SAM build, deploy) executing successfully on the `main` branch.

6.9.6 Database: Populated Corpus from Five Sources

Figure 6.6 shows the pgAdmin interface with the `documents` table containing ingested records from all five data sources, demonstrating that the ingestion pipeline successfully populates the corpus.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open pgAdmin at `http://localhost:5050` (login: see `docker-compose.yml`)
- Navigate to: **Servers** → **techpulse_db** → **Schemas** → **public** → **Tables** → **documents**
- Right-click `documents` → **View/Edit Data** → **All Rows**
- Run ingestion first if table is empty: `docker exec -it scheduler python ingest.py`
- The table grid must show **rows from all 5 sources** visible in the `source` column:
  `arxiv`, `hn`, `devto`, `github`, `rss`
- Visible columns must include: `id`, `source`, `title`, `state`, `published_at`
- Scroll or resize so the `source` column is clearly readable
- Capture both the pgAdmin left-panel tree (showing database name) and the data grid

Quick query to verify all 5 sources present:

```
SELECT source, COUNT(*) FROM documents GROUP BY source ORDER BY source;
```

Save as: `report/figures/demo_pgadmin.png` (PNG, min 1400px wide to show both panels)

Figure 6.6. pgAdmin: the `documents` table populated with records from all five data sources (ArXiv, Hacker News, DEV.to, GitHub Trending, RSS), demonstrating successful multi-source ingestion.

Chapter Summary. This chapter documented the implementation status across all 20 system components using a three-tier status classification (unit-tested, integration-tested, cloud-validated), deviations from the original proposal with honest trade-off analysis, key engineering observations from development, and the evaluation framework status. All core pipeline components are integration-tested via Docker Compose; cloud deployment is in progress across both AWS environments. Chapter 7 presents the remaining work plan, known limitations, and project timeline.

Team Contributions. Aye Khin Khin Hpone led data ingestion, preprocessing, embedding, and the React frontend. Dechathon Niamsa-Ard led the RAG orchestrator, LLM fallback chain,

AWS infrastructure (SAM templates), CI/CD pipeline, and evaluation framework. Both members contributed to the system architecture design, unit testing, and report writing.

CHAPTER 7

REMAINING WORK AND LIMITATIONS

7.1 Remaining Work

The remaining work is organized into four phases spanning from mid-March through the second week of April 2026. Table 7.1 provides a phase-level overview, followed by detailed task descriptions.

Table 7.1. Remaining Work: Phase Overview.

Phase	Focus	Timeline	Key Deliverables
A	System Validation & Corpus Building	Mar W3–W4	Populated corpus; local integration verified
B	Evaluation & Experimentation	Mar W4–Apr W1	Empirical results; grid search output
C	Analysis & Monitoring Validation	Apr W1–W2	Drift validation; results visualization
D	Final Report & Submission	Apr W2	Completed report with empirical evidence

7.1.1 Phase A: System Validation and Corpus Building (Mar W3 – Mar W4)

1. **Corpus population via multi-cycle ingestion.** Execute scheduled ingestion cycles across all five data sources over a sustained period to build a sufficiently large and diverse corpus. Target: 1,000+ indexed documents with balanced source representation across ArXiv, Hacker News, DEV.to, GitHub Trending, and RSS feeds.

2. **Local Docker Compose integration testing.** Verify end-to-end pipeline functionality in the local Docker Compose environment: ingestion → preprocessing → embedding → indexing → retrieval → generation. Confirm all six services (db, pgadmin, localstack, api, frontend, scheduler) operate correctly together.

3. **Frontend integration verification.** Test the React frontend against the live API to verify citation-grounded response rendering, baseline/hybrid mode toggling, abstention message handling, source badge display, and retrieval metadata transparency.

4. **Cloud deployment (Learner Lab and Free Tier, in progress).** Deploy the full system to both cloud environments. The Learner Lab deployment uses `template-learnerlab.yaml` and `samconfig-learnerlab.toml`; the Free Tier deployment uses `template.yaml` and `samconfig.toml`, enabling Amazon Bedrock as

the primary LLM backend. Both SAM templates have been prepared and validated, and initial stack creation is underway. Remaining: verify Lambda function execution, RDS connectivity, S3 storage operations, SQS message flow, and API Gateway routing under cloud conditions in each environment.

7.1.2 Phase B: Evaluation and Experimentation (Mar W4 – Apr W1)

5. **Full baseline evaluation run.** Execute 25 evaluation queries in vector-only (baseline) mode against the populated corpus. Record context precision, faithfulness, citation grounding ratio, and per-query latency for each query.

6. **Full hybrid evaluation run.** Execute the same 25 evaluation queries in hybrid mode using the default reranking weights ($\alpha = 0.6$, $\beta = 0.2$, $\gamma = 0.2$). Record all metrics under identical corpus and embedding conditions to enable paired comparison.

7. **Grid search weight optimization.** Run the reranking weight grid search across $\alpha \in \{0.4, 0.5, 0.6, 0.7\}$ with the latency constraint (p95 $\leq$ 2.0s). Identify the optimal weight configuration that maximizes mean context precision.

8. **Latency profiling and cost estimation.** Decompose end-to-end latency by stage (metadata filtering, vector search, reranking, LLM generation). Compute per-query cost estimates based on measured token usage across LLM backends.

7.1.3 Phase C: Analysis and Monitoring Validation (Apr W1 – Apr W2)

9. **Drift detection validation.** Execute multiple drift detection cycles against the populated corpus to validate the dual-criteria mechanism (10% relative threshold + Shewhart 3σ). Confirm that alerts trigger correctly under simulated drift conditions and that baseline history accumulates as expected.

10. **Results analysis and visualization.** Produce comparison tables and visualizations: baseline vs. hybrid precision paired differences, grid search Pareto frontier (precision vs. latency), per-source retrieval distribution, and latency breakdown charts.

11. **System demonstration screenshots.** Capture screenshots for all placeholder figures in the report (Figures 6.1–6.6 and Appendix C), replacing placeholder boxes with actual system output from Docker Compose, Learner Lab, and Free Tier environments.

7.1.4 Phase D: Final Report and Submission (Apr W2)

12. **Evaluation chapter completion.** Write the complete evaluation results chapter with empirical findings, statistical comparison of baseline vs. hybrid retrieval, discussion of grid search outcomes, and analysis against the four evaluation targets (precision > 0.75, high faithfulness, p95 $<$ 2s, cost $<$ USD 15/month).

13. **Discussion and conclusions.** Complete the discussion of results including the five planned analyses (Section 5.5): precision vs. overhead, retrieval strictness vs. recall, cost–performance balance, weight sensitivity, and production suitability assessment.

14. **Final report revision and submission.** Integrate all empirical content, screenshots, and analysis into the final report. Conduct internal peer review, address formatting and consistency, and prepare the submission package including source code archive.

7.2 Known Limitations

1. The evaluation corpus is limited to five technology-focused data sources; generalizability to other domains or larger corpora requires further study.

2. The evaluation query set of 25 curated queries is sufficient for preliminary comparative analysis consistent with prior RAG evaluation practices (Es et al., 2023). However, a larger and more diverse query set would strengthen external validity.

3. Amazon Bedrock is unavailable in the Learner Lab environment, requiring evaluation with HuggingFace Inference API and Ollama under that configuration. However, the system has also been deployed on the AWS Free Tier, where Bedrock (Claude Haiku) is available as the primary backend. Since the prompt template, evaluation methodology, and RAGAS metrics are backend-agnostic, the comparative findings between baseline and hybrid retrieval remain valid and transferable across both environments. A supplementary per-backend faithfulness comparison is planned during Phase B to quantify the model quality difference.

4. The `db.t3.micro` RDS instance provides limited compute and memory compared to Aurora Serverless. Under high ingestion volumes, database performance may become a bottleneck.

5. Cross-encoder reranking and ANN-optimized indexing strategies are out of scope for this project but identified as future research directions.

6. Budget governance via Cost Explorer has higher latency than the originally proposed AWS Budgets integration, as Cost Explorer data may lag by up to 24 hours.

7. The current unit test suite (49 tests) covers core logic but lacks integration tests that exercise the full pipeline end-to-end with a live database and AWS services.

8. **Context window management.** The current implementation retrieves top-$k = 8$ chunks at 500 tokens each, consuming approximately 4,100 prompt tokens. For Mistral-7B on HuggingFace (4,096-token context window), this leaves only $\sim$few hundred tokens for system instruction and generation. If retrieved chunks are densely filled, the context may be truncated silently. A token budget check before prompt construction is identified as a near-term improvement.

9. **Secrets management.** API keys (HuggingFace token, GitHub token) are currently stored as environment variables in Lambda configurations. For production hardening, migration to AWS Secrets Manager or SSM Parameter Store is recommended to avoid key exposure in infrastructure-as-code templates.

7.3 Project Timeline

Figure 7.1 presents the project timeline, including completed milestones and remaining work.

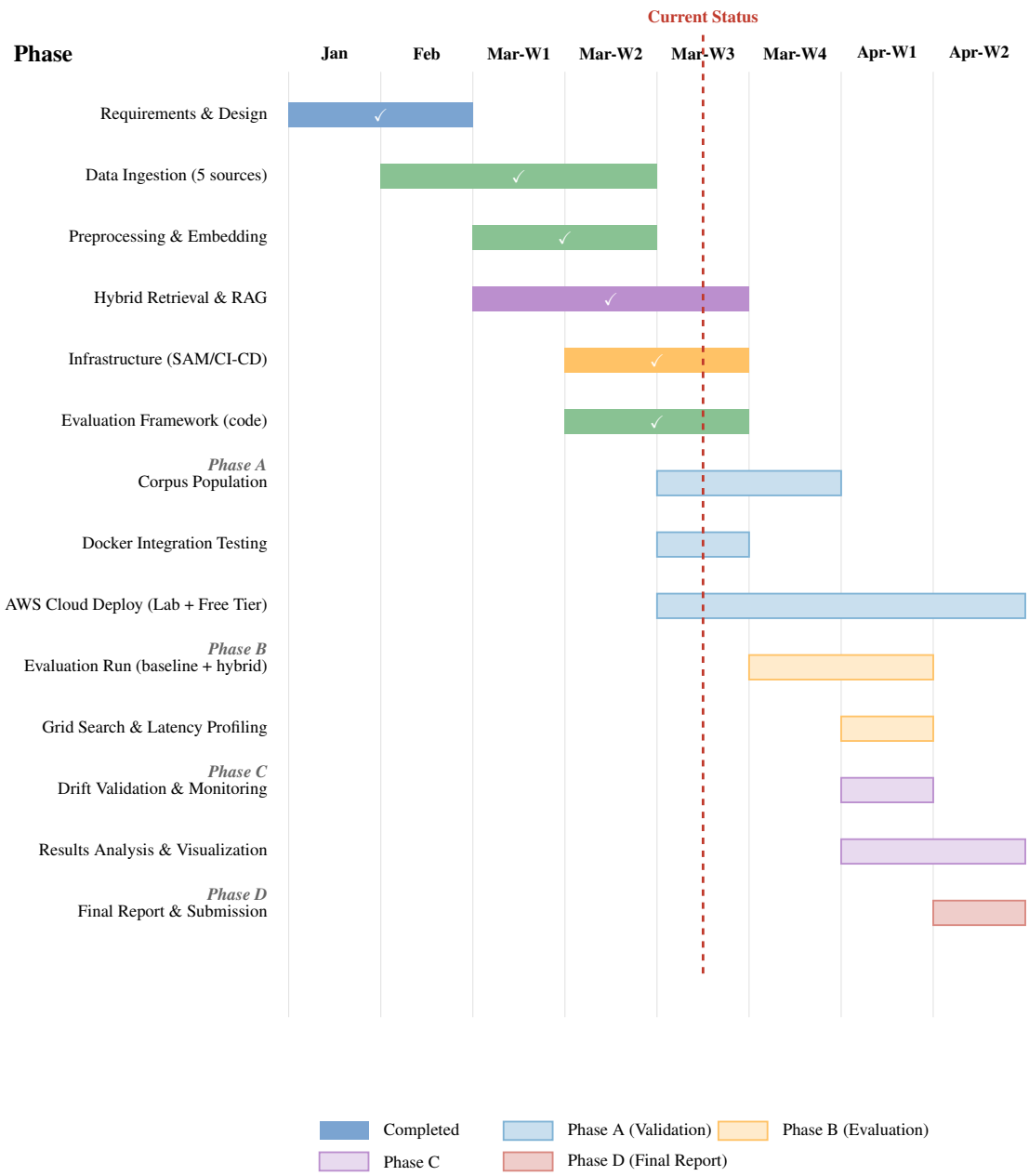

Figure 7.1. Project timeline (Gantt chart). Solid bars indicate completed phases; outlined bars indicate remaining work organized into four phases (A–D). The dashed vertical line marks the progress report submission. All remaining work is scheduled to complete by second week of April 2026.

7.4 Team Contributions

Table 7.2 summarises the primary responsibilities of each team member.

Table 7.2. Team Member Contribution Summary.

Component	Primary	Contributor
Data Ingestion (5 sources)	Aye Khin Khin Hpone	Dechathon Niamsa-Ard
Preprocessing & Chunking	Dechathon Niamsa-Ard	Aye Khin Khin Hpone
Embedding & Storage	Dechathon Niamsa-Ard	Aye Khin Khin Hpone
Hybrid Retrieval & RAG	Aye Khin Khin Hpone	Dechathon Niamsa-Ard
LLM Backend Fallback	Dechathon Niamsa-Ard	Aye Khin Khin Hpone
Drift Detection & Monitoring	Aye Khin Khin Hpone	Dechathon Niamsa-Ard
Infrastructure (SAM/CI-CD)	Dechathon Niamsa-Ard	Aye Khin Khin Hpone
Evaluation Framework	Aye Khin Khin Hpone	Dechathon Niamsa-Ard
Frontend (React)	Dechathon Niamsa-Ard	Aye Khin Khin Hpone
Unit Tests (49 tests)	Both	—
Documentation & Report	Both	—

Both team members participated in architectural design decisions, code review, and debugging throughout the project.

7.5 Future Extensions

Several extensions are identified for post-prototype development:

- Migration to Aurora Serverless (already feasible in the Free Tier environment; applicable when moving beyond Learner Lab constraints)
- Full evaluation with Amazon Bedrock (Claude Haiku) as the primary backend via the Free Tier deployment
- Expansion of data sources (StackExchange, academic conference proceedings)
- Cross-encoder reranking evaluation for precision-critical use cases
- Embedding model fine-tuning or replacement based on drift analysis findings
- Integration testing suite with containerized database and mocked AWS services
- User feedback loop for continuous retrieval quality improvement

- Token budget management: pre-prompt token count check to prevent context truncation in constrained-context-window backends
- Secrets Manager / SSM Parameter Store migration for production-grade API key handling

Chapter Summary. This chapter presented the structured four-phase completion plan (March W3 through April W2), nine known limitations with honest assessments, the project Gantt chart, and seven future extension directions. The primary remaining risks are cloud deployment verification and the full evaluation run; both are mitigated by the dual-environment setup (Learner Lab + Free Tier) and the pre-built evaluation codebase. The final report will integrate empirical results, screenshots, and analysis completing the evidence base for the four research questions stated in Chapter 1.

REFERENCES

AWS. (2023). *AWS well-architected framework*. Amazon Web Services. https://aws.amazon.com/architecture/well-architected/

Bayram, F., Ahmed, B. S., & Kassler, A. (2022). From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems*, *245*, 108632. https://doi.org/10.1016/j.knosys.2022.108632

Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., de Las Casas, D., Guy, A., Menick, J., Ring, R., Hennigan, T., Huang, S., Maggiore, L., Jones, C., Cassirer, A., . . . Sifre, L. (2022). Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning* (Vol. 162, pp. 2206–2240). PMLR.

Breck, E., Polyzotis, N., Roy, S., Whang, S., & Zinkevich, M. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. In *Proceedings of the IEEE International Conference on Big Data* (pp. 1123–1132). IEEE.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., . . . Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901.

Es, S., James, J., Anke, L. E., & Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, *46*(4), 1–37. https://doi.org/10.1145/2523813

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., & Grave, E. (2022). Atlas: Few-shot learning with retrieval augmented language models. *Advances in Neural Information Processing Systems*, *35*, 22251–22265.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12), 1–38. https://doi.org/10.1145/3571730

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, *31*(12), 2346–2363. `https://doi.org/10.1109/TKDE.2018.2876857`

Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 9004–9017). Association for Computational Linguistics.

Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-T., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScoring: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 12076–12100). Association for Computational Linguistics.

Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2023). ARES: An automated evaluation framework for retrieval-augmented generation systems. *arXiv preprint arXiv:2311.09476*.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, *28*, 2503–2511.

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., & Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning* (pp. 31210–31227). PMLR.

Thakur, N., Reimers, N., Daxenberger, J., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the 35th Conference on Neural Information Processing Systems—Datasets and Benchmarks Track*.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, *33*, 5776–5788.

Zhang, T., Chen, Y., & Zhou, Z. (2020). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Software*, *37*(6), 1–17.

Zhang, Y., Chen, X., Zhang, Y., Hu, X., Liu, Z., & Tang, J. (2024). A comprehensive survey on retrieval-augmented generation. *ACM Transactions on Information Systems*. Advance online publication. https://doi.org/10.1145/3703155

APPENDIX A

SOURCE CODE AVAILABILITY

The implementation of *TechPulse: Hybrid Retrieval-Augmented Generation System for Emerging Technology Intelligence* has been developed using Python 3.11 and modern cloud-native technologies.

To support reproducibility, transparency, and collaborative development, the project source code is maintained in a GitHub organization repository:

`https://github.com/orgs/EZ2-555555/repositories`

The repository includes the following implemented components:

- Five data ingestion modules for collecting data from ArXiv API, Hacker News API, DEV.to API, GitHub Trending API, and curated RSS/Atom feeds
- A 17-step preprocessing pipeline for comprehensive text normalization and token-based chunking using tiktoken (`cl100k_base`)
- Semantic embedding pipeline using the `all-MiniLM-L6-v2` sentence-transformer model (384-dimensional embeddings)
- Hybrid retrieval implementation including metadata filtering, cosine similarity search via pgvector, and reranking-lite scoring with configurable weights
- RAG orchestration with three-layer hallucination verification (prompt grounding, RAGAS faithfulness, citation traceability)
- Automatic LLM backend fallback chain supporting Bedrock, Ollama, and HuggingFace Inference API
- Dual-criteria drift detection combining Shewhart 3σ control charts with relative-drop thresholds
- Infrastructure-as-Code definitions using AWS SAM for both Learner Lab and Free Tier deployment (15+ AWS resources per environment)
- React-based frontend interface for query submission and citation-grounded response display
- Evaluation framework with 25 curated queries, RAGAS metrics, and grid search optimization
- Four-stage CI/CD pipeline using GitHub Actions (lint, test, build, deploy)

- 49 unit tests across 5 test modules with full pass status
- Docker Compose configuration for local development with 6 services

APPENDIX B

REPOSITORY STRUCTURE

The project repository is organized as follows:

```
data-pipeline/
  src/
    config.py              # Centralized configuration
       (30+ settings)
    scheduler.py           # EventBridge schedule
       definitions
    api/main.py            # FastAPI + Mangum (/health, /
       ask, /drift)
    db/
      connection.py        # PostgreSQL connection pool
      init_schema.py       # Schema + pgvector + HNSW
         index
    embedding/embedder.py  # MiniLM singleton embedder
    ingestion/
      arxiv_ingester.py    # ArXiv API (cs.AI, cs.LG, cs.
         CL, cs.IR, cs.SE)
      hn_ingester.py       # Hacker News Firebase API
      devto_ingester.py    # DEV.to REST API (21 tags)
      github_ingester.py   # GitHub Search API (10 topics)
      rss_ingester.py      # 10 curated RSS/Atom feeds
    observability/
      __init__.py          # Deep health check (4 probes)
      drift.py             # Dual-criteria drift detection
    orchestrator/
      llm_backends.py      # 3-backend auto-fallback chain
      rag.py               # RAG pipeline + hallucination
         verification
    pipeline/run_pipeline.py  # End-to-end pipeline
       orchestrator
    preprocessing/chunker.py  # 17-step normalize + token
       chunking
    retrieval/retriever.py    # Baseline + hybrid 3-stage
       retrieval
    queue/                    # SQS producer/consumer
```

```
  storage/                    # S3 medallion operations
tests/                  # 49 unit tests (5 modules)
evaluation/             # RAGAS evaluation + grid search
infra/
  template-learnerlab.yaml  # SAM template (Learner Lab)
  samconfig-learnerlab.toml # SAM deployment config (
     Learner Lab)
  template.yaml             # SAM template (Free Tier)
  samconfig.toml            # SAM deployment config (
     Free Tier)
frontend/               # React SPA
docker-compose.yml      # 6 local services
.github/workflows/ci.yml   # 4-stage CI/CD
```

APPENDIX C

SYSTEM DEMONSTRATION SCREENSHOTS

This appendix provides supplementary screenshots demonstrating additional system functionality.

Health Check Endpoint

Figure C.1 shows the `/health` API endpoint response, confirming connectivity to all four service dependencies (PostgreSQL, S3, SQS, and LLM backend).

Screenshot: Browser or curl output of `GET /health` showing DB, S3, SQS, and LLM status.

File to place: `figures/demo_health.png`

Figure C.1. Health check endpoint (`/health`): JSON response confirming database connectivity, document/chunk counts, S3 reachability, SQS reachability, and LLM backend status.

Drift Detection Endpoint

Figure C.2 shows the `/drift` API endpoint returning a drift detection report, demonstrating the dual-criteria Shewhart 3σ monitoring mechanism.

Screenshot: Browser or curl output of `GET /drift` showing mean similarity, alert status, and probe results.

File to place: `figures/demo_drift.png`

Figure C.2. Drift detection endpoint (`/drift`): JSON response showing probe query mean similarity, standard deviation, alert status, and Shewhart 3σ control analysis.

API: RAG Query Response

Figure C.3 shows the raw JSON response from the `/ask` endpoint, displaying the structured output including the generated answer, source metadata, similarity scores, and retrieval mode.

Screenshot: Browser or curl output of `POST /ask` showing full JSON response with answer, sources, and scores.

File to place: `figures/demo_api.png`

Figure C.3. RAG query endpoint (`POST /ask`): full JSON response showing the generated answer, retrieved source documents with titles, URLs, similarity scores, and retrieval mode metadata.